UniRL: Trajectory-Centric Post-Training Across Heterogeneous Generative Models

UniRL Authors *authors@to.be.added*
Affiliations to be added before the arXiv release

Abstract

Reinforcement-learning post-training for generative models follows a common semantic loop—generate, score, assign credit, and update—yet existing systems fragment across autoregressive, diffusion, and multi-stage multimodal workflows, as well as across colocated, disaggregated, and asynchronous execution topologies. We present UniRL, a trajectory-centric system that places a lineage-preserving, modality-aware intermediate representation (IR) at the boundary between post-training semantics and distributed execution. Each trajectory is represented as a prompt-rooted tree of typed layers, where each generated layer records its decoded primitives, conditioning inputs, stage-specific replay segment, rewards, advantages, and behavior-policy version. Three logical roles operate on this representation—rollout engines populate generation layers, reward and credit operators evaluate and fold feedback along tree ancestry, and training backends replay stage-specific objectives—while explicit runtime contracts govern device placement, tensor transport, weight publication, and memory residency. These interfaces decouple model-specific replay from lineage tracking, data movement, and scheduling, without forcing heterogeneous objectives or engine–topology pairs into a monolithic trainer. We implement this design in UniRL v0.2.0, validate ten core IR invariants in a source-importing CPU harness, and examine its reward-serving and rollout-engine boundaries through two image-editing case studies on generation–reward overlap and conditioning fidelity. Across three seeds, SD3.5-Medium diffusion post-training improves held-out automatic preference and compositional scores at the cost of reduced within-prompt diversity (−24.78% LPIPS), while in an eight-GPU native-recipe benchmark, UniRL achieves lower per-iteration time than VeRL-Omni (64.22 s vs. 111.33 s). In a recipe-matched 500-rollout Qwen3-4B-Base Group Relative Policy Optimization (GRPO) comparison, UniRL records 8.2% higher end-to-end output-token throughput than veRL on runs executed concurrently on separate cluster nodes, with UniRL sampling 11.0% longer responses and a higher 8,192-token truncation rate (0.307 vs. 0.203). On the primary autoregressive endpoint (MATH-500), the prespecified GRPO recipe improves at early checkpoints but degrades by the final 800-rollout checkpoint, ending below the frozen base model across all three seeds; a post-hoc DRPO recipe finishes above the base model across all three seeds under standard grading, though one seed remains above the base only when responses that reach the generation token limit are graded as valid. In a composed rewrite-to-image workflow, two lineage-defined groupings match an ID-joined reference on recorded rollouts and both improve over the base model in training, while their three-seed quality contrast is inconclusive. Throughout the evaluation, we distinguish metrics computed directly from exported run artifacts (weight-publication parity, autoregressive and diffusion training, cross-stage grouping, and the rerun autoregressive systems benchmark) from legacy report-derived summaries: isolating the cause of late-stage GRPO degradation, verifying replay-tensor lineage on recorded rollouts, measuring paired IR representation overhead, and establishing a controlled time-to-quality comparison remain open.

1 Introduction

Online post-training is a feedback program: it expands inputs into sampled trajectories, evaluates those trajectories, turns feedback into a learning signal, and applies one or more optimizer updates. At this level of abstraction, autoregressive (AR) text generation, diffusion denoising, and multi-stage multimodal pipelines look nearly identical, yet their underlying data paths and state requirements diverge sharply. An AR update may need packed token IDs, masks, rollout log probabilities, and per-token critic value estimates; a diffusion update may need a sparse set of latent states, a noise schedule, and transition statistics; a composed workflow must retain which image descended from which rewritten prompt; and an asynchronous run must additionally track which behavior version generated each trajectory layer.

Existing systems have made major progress on scalable LLM post-training and flexible device mapping (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025). Fully or partially asynchronous systems further address rollout tail latency and accelerator idle time (Fu et al., 2025; Zhang et al., 2026; Zhu et al., 2025). Post-training is now expanding from single-stage text generation to iterative diffusion denoising and multi-stage multimodal pipelines (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025; Huang et al., 2026). In multi-stage and multimodal workflows, consecutive generation stages differ in modality, replay state, and branching factor while sharing tree ancestry for conditioning and credit assignment, making flat, LLM-specific tensor dictionaries insufficient. The central systems question is how to preserve shared trajectory structure—lineage, conditioning, credit, and policy provenance—across heterogeneous model states and distributed execution topologies:

> *Can a single post-training programming model span heterogeneous generative model families and execution topologies while preserving the trajectory structure required for grouping, replay, credit assignment, tensor transport, and behavior-version control?*

Our answer in UniRL is to introduce a *trajectory intermediate representation (IR)* that captures multi-stage lineage, modality-specific state, and policy provenance across stages with differing batch sizes, and to expose physical execution choices as explicit contracts around that IR. Rather than treating an individual row or a flat LLM-specific tensor dictionary as the unit of scheduling and sharding, the IR organizes each batch as a forest of prompt-rooted trees whose width expands whenever a stage branches. Within each tree layer, typed containers separate decoded modality primitives, model-facing conditions, and stage-specific replay segments. Distributed methods declare how each structured field scatters, concatenates, packs, or remains shared, and dense tensors can be replaced by row-addressable references without changing the trajectory structure. This representation flows through a small set of logical roles that can be mapped onto colocated or separate *GPU slabs* (contiguous subsets of accelerators allocated to a role).

The paper develops three coupled design ideas:

- A *lineage-preserving trajectory IR* in which a rollout request is a typed tree skeleton spanning AR and diffusion stages, each generated layer is conditioned through its ancestry and retains its own replay segment, and training folds reward and credit back along the same tree (Sections 3.1–3.5). Synthetic fixture checks and an ID-join check on recorded rollouts of a composed prompt-rewrite-to-image run verify the grouping and advantage semantics, while the downstream quality comparison between the two groupings is inconclusive and replay-tensor verification on recorded rollouts remains open.
- A *modular interface for stage replay and optimization* that decouples trajectory layers from stage-specific objectives and distributed training backends, reducing the integration complexity of N_{mod} model bundles and N_{alg} shared algorithms from $N_{\text{mod}} \times N_{\text{alg}}$ to $N_{\text{mod}} + N_{\text{alg}}$ (Sections 3.6 and 5).
- A *factorization of semantic roles from physical execution*: rollout engines implement a uniform skeleton-filling transformation (`Sample` $\rightarrow$ `Sample`, where `Sample` is the batched trajectory container), while field-aware dispatch, tensor references, placement scopes, weight publication, and memory residency govern execution across colocated, disaggregated, and asynchronous topologies (Section 4), complemented by case studies on reward-serving isolation and overlap and on rollout-engine conditioning fidelity (Sections 6 and 7). Pre-launch and in-run parity checks across five weight-publication paths, together with native-

recipe diffusion, recipe-matched AR, and deployment-topology benchmarks, evaluate these boundaries; paired IR representation overhead remains unmeasured.

Asynchronous execution is where the third idea pays off most directly. Because rollout, reward, and training are logical roles rather than fixed steps of one loop, the orchestration layer—admission, root-complete grouping, version stamping, lag filtering, and weight publication—can be lifted out of individual trainers. UniRL's AR and diffusion trainers therefore share one bounded-staleness manager over the same trajectory IR in code (Section 4.5). Empirically, our exploratory bounded-staleness study exercises only the Qwen3-4B-Base AR workload: single-seed planning runs with evaluation disabled record lower per-rollout time once generation–training overlap is enabled, but the zero-lag disaggregated quality-target pilot degraded between rollouts 150 and 300 and the formal sweep was canceled, so no asynchronous time-to-quality or diffusion-asynchrony result is claimed (Section 8.4).

The evidence is organized from structural correctness, through credible learning and a composed workflow, to measured system cost. The experiment IDs and corresponding artifacts are specified in Section 8.

The current evidence comprises a CPU IR regression suite, GPU weight-publication parity checks (E0), three-seed training evaluations on autoregressive math reasoning (E1) and diffusion image generation (E2) alongside single-seed HunyuanImage-3.0 and MiniMax-H3 recipe reproductions, a three-seed composed rewrite-to-image grouping study (E4), an eight-GPU native-recipe diffusion benchmark (E3) and deployment-topology comparison (E5-T), and a recipe-matched 16-GPU autoregressive systems comparison (E3-AR). Each result carries explicit scope boundaries: E2 improves held-out preference and composition scores but incurs a -24.78% within-prompt LPIPS diversity reduction; on E1's registered MATH-500 endpoint, the prespecified GRPO recipe finishes below the frozen base model in all three seeds, whereas a post-hoc DRPO recipe finishes above the base in all three seeds as graded (with one seed relying on responses that reach the generation token limit); E4 verifies lineage grouping and advantages against an ID-joined reference on recorded rollouts, but its three-seed quality comparison between root and rewrite grouping is inconclusive; and in systems benchmarking, E3 retains estimator and LoRA-target differences while E3-AR comprises one process-level run per framework on separate cluster nodes with diverging response lengths and truncation rates. Consequently, these results do not establish a cross-workload quality or time-to-quality advantage, and completing the trajectory-centric evaluation still requires replay-tensor validation on recorded rollouts (E4a) and a paired measurement of IR representation overhead (E5-R).

2 Problem: Preserve Semantics While Changing Execution

Let an online post-training iteration consume a batch of initial root inputs X_0 (such as text prompts or multimodal contexts), generate a trajectory batch $\mathcal{T}$ under behavior policy π_v, evaluate rewards R via reward operator $\mathcal{R}$, compute credit A via credit-assignment operator $\mathcal{G}$, and update policy parameters θ via update operator U_θ:

$$X_0 \xrightarrow{\pi_v} \mathcal{T} \xrightarrow{\mathcal{R}} (\mathcal{T}, R) \xrightarrow{\mathcal{G}} (\mathcal{T}, A) \xrightarrow{U_\theta} \theta'. \tag{1}$$

The compact notation hides three forms of heterogeneity.

Trajectory heterogeneity. The sufficient statistics required for a policy update depend on the generative process. For AR policies, a replay segment typically stores variable-length packed token sequences, attention and loss masks, rollout log probabilities, and per-token critic value estimates or returns. For stochastic differential equation (SDE) diffusion or flow policies, a replay segment instead stores latent snapshots at selected denoising steps, a shared noise schedule, the indices of sampled transitions, and rollout transition statistics (such as step log probabilities or transition means). Multi-stage pipelines and multi-turn agentic workflows further introduce branching lineage and interleaved environment observations.

For example, suppose each root prompt p branches into k_1 text rewrites $w_{p,j}$, and each rewrite branches into k_2 images $x_{p,j,k}$. Although the image generator conditions directly on $w_{p,j}$, credit assignment $\mathcal{G}$ may normalize rewards either across the k_2 images sharing rewrite $w_{p,j}$ or across all $k_1 k_2$ images descending from root prompt p. Supporting both grouping scopes over a flattened image batch requires explicitly retaining or reconstructing the parent and root identifiers (p, j) for every row. This branching structure motivates

explicit lineage tracking, whereas the diffusion model's denoising state independently motivates typed replay segments; neither mechanism subsumes the other.

Role heterogeneity. Generation, reward evaluation, replay, gradient computation, and weight publication differ in runtime, parallel layout, and lifecycle, not only in kernels. Rollout may run inside the training process or in a dedicated serving engine with its own tensor-parallel (TP) layout, such as SGLang (Zheng et al., 2024), vLLM (Kwon et al., 2023), or vLLM-Omni (Yin et al., 2026), while a sharded training backend such as FSDP (Zhao et al., 2023) or VeOmni (Ma et al., 2025a) owns parameters and optimizer state. Reward ranges from an in-process scorer to an environment-isolated scorer process or a remote multi-model service. These roles must agree on when each copy of the policy is resident, offloaded, or published, and publishing from a sharded trainer is itself a collective over the whole training mesh. Which boundary dominates cost depends on the configuration: in the measured separate-slab deployment, much of the slowdown arises from reward processing (Section 8.3).

Topology heterogeneity. A small-scale run may colocate and time-share training and rollout on a single GPU allocation; a larger deployment may disaggregate them onto separate GPU slabs; and workloads with variable generation lengths may overlap rollout and training via bounded-staleness asynchronous admission. Each choice changes memory pressure, transfer paths, synchronization frequency, and allowable policy lag.

These observations produce four requirements (Table 1). The first two follow from trajectory heterogeneity and the third from role and topology heterogeneity; the fourth specializes the third to asynchronous execution, where decoupled roles may train on work generated by an older policy.

Table 1: Design requirements and the corresponding UniRL mechanism. The right column states what must be evaluated; it is not a result.

Requirement	Mechanism	Evidence required
Preserve lineage	ID-encoded ancestry validated at construction; split, select, concat, data-parallel sharding, and reward propagation over complete root groups	Trace invariants across AR, diffusion, and cross-stage runs against an ID-joined reference
Preserve replay inputs	Typed primitives and conditions; packed AR and sparse latent replay segments; σ-schedule echo checks on external diffusion engines	Rollout–replay consistency on exercised model paths
Change execution safely	Logical roles, placement scopes, field-aware dispatch, tensor references, weight publication and residency lifecycle, and startup rejection of unvalidated combinations	Replay parity after each publication path; matched-work topology and transport ablations
Control policy age	Per-layer behavior versions, complete-group admission, lag filters, quiescent publication, mixed-version rejection, and hard boundaries	Lag distributions, throughput, and time-to-quality

Non-goals. UniRL does not eliminate model-specific code; instead, it isolates architecture-specific forward passes, conditioning encoders, and rollout adapters within a *model bundle*, which interfaces with stage-level algorithms that compute losses over replayed segments. Nor does UniRL automatically search for optimal device placements or replace the underlying cluster scheduler: the runtime uses a central driver process to orchestrate logical roles and Ray actors (Moritz et al., 2018) to host distributed workers.

3 A Trajectory-Centric Programming Model

3.1 A trajectory batch is a sequence of tree layers

The central data type is a trajectory batch

$$\mathcal{T} = [P_0, P_1, \ldots, P_L], \qquad P_i = (I_i, X_i, C_i, Z_i, R_i, A_i, V_i, M_i), \tag{2}$$

where, for each depth i, I_i stores row identifiers, X_i decoded multimodal primitives, C_i model conditions, Z_i stage-specific replay segments, R_i rewards, A_i advantages or returns, V_i the behavior-policy version v_b stamped by the generating engine (Section 4.3), and M_i auxiliary metadata. In code, $\mathcal{T}$ is an instance of `Sample` and each P_i is an instance of the batched class `Part`; we refer to P_i as a *tree layer* (or simply *layer*) throughout the paper. The sequence $[P_0, \ldots, P_L]$ encodes a forest of prompt-rooted trees in which P_i batches all nodes at depth i and lineage edges connect parent rows in P_{i-1} to child rows in P_i. We call the deepest layer P_L the *frontier*; a trajectory batch grows by appending a new layer after the current frontier. Each generated layer is parameterized by the stage sampling configuration that governs its generation and carries a corresponding replay segment—such as a `TextSegment` for an AR layer or a `LatentSegment` for a diffusion layer—whereas input layers carry root prompts or environment observations that condition subsequent generation.

Row identifiers encode tree ancestry directly: root IDs in I_0 contain no lineage separator, and each branching step appends a delimiter and child ordinal (e.g., $p_1/0$). For every non-root layer P_i ($i > 0$) and every row identifier $s \in I_i$, the parent identifier must exist in I_{i-1}:

$$\forall i > 0,\ \forall s \in I_i : \quad \text{parent}(s) \in I_{i-1}. \tag{3}$$

The `Sample` constructor enforces Equation 3 whenever a batch is built or extended. Two structural operators grow the tree at the frontier: `fork` appends a child generation layer with k child rows per frontier row, and `observe` appends a one-to-one ($k = 1$) environment observation layer. In the reverse direction, `propagate_rewards` folds rewards from a scored child layer P_i back onto an unscored parent layer P_{i-1}.

Batch-level partitioning and indexing operate on complete root trees rather than individual rows: `Sample.split()` decomposes a batch into one tree-complete `Sample` per root ID, and slicing or subset selection acts over these root groups. Manipulating whole root trees preserves the parent-major sibling contiguity established by `fork`, whereas permuting child rows independently can break the contiguous sibling blocks assumed by group-based reward reduction and advantage estimation.

Well-formed batches. Lineage operations require well-formed trajectory batches in which (i) root IDs in I_0 are unique and (ii) within every non-root layer P_i, sibling rows appear in contiguous parent-major order; when concatenating multiple batches, they must additionally share the same layer depth L and compatible per-layer field metadata. The programming model maintains these invariants by construction—growing trajectories exclusively through `fork` and `observe` and partitioning batches only along root-group boundaries—so reward reduction and group-advantage operators can scan each sibling group as a single contiguous slice. At runtime, group-advantage computation explicitly rejects non-contiguous sibling layouts, shard concatenation verifies matching layer metadata, and the asynchronous batch assembler enforces homogeneous behavior versions v_b within each admitted batch (validated by our structural regression suite in Appendix D).

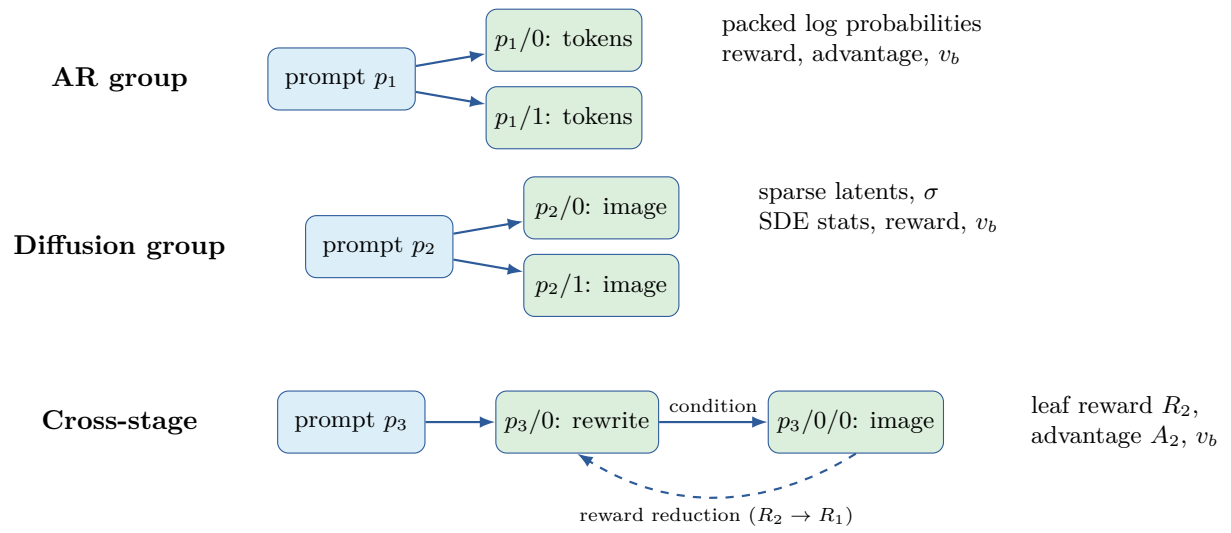


Figure 1: Three trajectory shapes represented by the same ordered-layer IR. Each generated layer retains decoded primitives, model conditions, and the stage-specific replay segment required for policy replay, alongside rewards, advantages, and the behavior version v_b. In the cross-stage path, lineage links the image layer to its parent rewrite so leaf rewards can fold back to upstream stages (or remain at the leaf when upstream stages are frozen).

3.2 Primitives, conditions, and replay segments are separate layer fields

Within each layer P_i, UniRL separates three payload components—decoded primitives X_i, model conditions C_i, and replay segments Z_i—that existing post-training pipelines often conflate.

Primitives (X_i). Primitives hold decoded or externally supplied modality payloads: text strings, image tensors, video frames, audio waveforms, and external media references. Variable-resolution images and variable-duration video or audio payloads use packed storage accompanied by per-example shapes or cumulative sequence offsets. Primitives are consumed by reward evaluators, logging and visualization tools, and multi-turn conditioning, but need not be the low-level tensors replayed during gradient updates.

Conditions (C_i). Conditions store model-facing encodings, such as prompt embeddings or reference image latents. Separating C_i from X_i lets rollout engines and training stages exchange precomputed conditioning tensors through a typed contract without prescribing a single tokenizer, text encoder, or multimodal fusion architecture.

Replay segments (Z_i). Replay segments store the stage-specific sufficient statistics required to recompute policy likelihoods or transitions during training. For autoregressive stages, `TextSegment` stores packed token IDs, attention masks, and rollout log probabilities. For diffusion and flow stages, `LatentSegment` stores latent snapshots at a subset of denoising steps together with their step-index map, a shared σ-schedule, and optional SDE transition log probabilities or means. Recording a sparse step-index map allows diffusion objectives to replay only selected denoising transitions rather than materializing the full denoising trajectory during backpropagation.

This separation decouples the lifetime of decoded media from the lifetime of replay state: once reward evaluation and logging complete, the trainer can discard large decoded primitives X_i while retaining conditions C_i, replay segments Z_i, rewards R_i, and advantages A_i for the training update.

3.3 A field algebra defines structural batching

Every field in a structured batch declares a composition kind: *concatenated* along the batch dimension, *packed variable-length*, *shared* across rows (the default for unannotated fields), or *scalar-reduced* under concatenation via maximum, minimum, sum, or mean (used in the current runtime to reduce per-shard timing metadata by maximum). Structural operations—concatenation, root-group selection, slicing, and repetition—traverse nested objects recursively, transforming concatenated and packed fields while passing shared fields through unchanged; device transfers apply to all underlying tensors regardless of kind.

The same field algebra governs distributed argument scattering and return-value reconstruction across role workers (Section 4). When a method is invoked under data-parallel (DP) scatter, an input `Sample` is partitioned into complete root groups across the DP shards (requiring the root count to be divisible by the DP degree), while auxiliary arguments whose leading dimension does not match the split count are replicated to each shard. Upon completion, the results returned by the primary rank of each DP shard are merged layer-by-layer according to each field's declared kind; for broadcast lifecycle methods, the runtime instead returns the per-worker result list so callers expecting identical state can verify cross-worker agreement explicitly.

Declaring batching semantics at the field level eliminates hand-written tensor slicing and padding inside individual roles, while relying on the well-formed batch invariants of Section 3.1 to guarantee valid shard merges. Because recursive tree traversal and tensor-reference bookkeeping (Section 4) introduce non-trivial CPU overhead, we benchmark their cost directly in Section 8.3 rather than assuming the abstraction is free.

3.4 Rollout unfolds a declared trajectory tree

A rollout request is a trajectory batch that declares the skeleton of the tree to be generated. Starting from an initial root batch $[P_0]$, the caller appends one target generation layer per stage via `fork`; prior to execution by a rollout engine, we refer to each unfilled generation layer as a *shell*. Each shell pre-allocates its row count (k children per parent row), populates child identifiers I_i that encode their parent IDs in I_{i-1}, and attaches a stage-specific sampling-parameter object—such as s^{AR} or s^{diff}—that specifies the generative process to run. For example, the prompt-enhancement and unified-model trainers construct the two-stage skeleton

$$\texttt{Sample}([P_0]).\texttt{fork}(k_1, s^{\mathrm{AR}}).\texttt{fork}(k_2, s^{\mathrm{diff}}) = [P_0,\ P_1^{\mathrm{AR}},\ P_2^{\mathrm{diff}}], \tag{4}$$

where s^{AR} and s^{diff} denote the autoregressive and diffusion sampling parameters. For B root prompts, the Bk_1 text rows in P_1^{AR} and Bk_1k_2 image rows in P_2^{diff} have fixed lineage before any token or latent is sampled. Every rollout engine implements a skeleton-filling transformation

$$F_{\theta,e} : \mathcal{T}_{\mathrm{skeleton}} \longrightarrow \mathcal{T}_{\mathrm{filled}}, \tag{5}$$

parameterized by policy weights θ and generation runtime e (either a train-side model pipeline or an external serving engine). The operator $F_{\theta,e}$ preserves the skeleton's layer structure and row identifiers I_i, populates each target shell with decoded primitives X_i, optional conditions C_i, and a replay segment Z_i, and stamps each filled layer with the behavior version $V_i = v_b$ of the engine that generated it. Alongside $F_{\theta,e}$, the base engine interface exposes residency-lifecycle and weight-publication methods (Section 4.3) without constraining the runtime's internal serving scheduler or kernel API.

One trajectory spans autoregressive and diffusion stages. In a multi-stage workflow, a single trajectory batch captures all generation stages in sequence. When populating shell P_i, the rollout engine queries `conditioning_at` to walk each row's ancestor chain $P_0, \ldots, P_{i-1}$ and collect the modality-tagged primitives along that path. In the text-to-text-and-image modes of HunyuanImage-3.0 and BAGEL, for instance, the first stage fills the AR shell P_1^{AR} from the root prompt in P_0, and the diffusion stage then reads both the root prompt and the generated reasoning text directly through each image row's ancestry in P_2^{diff}, eliminating external side tables to join images with rewritten prompts. Because each stage writes its replay state into its own layer—a `TextSegment` in P_1^{AR} and a `LatentSegment` in P_2^{diff}—while sharing the same tree lineage I_0, I_1, I_2, the filled `Sample` retains both the ancestry required for cross-stage credit assignment and the stage-specific sufficient statistics required for policy replay (Section 3.5). An AR stage and a diffusion stage are

thus represented as consecutive typed layers of one prompt-rooted tree rather than separate batches joined after generation. Using this single representation, our trainers express single-stage AR, single-stage diffusion, two-stage AR-to-diffusion, and multi-turn tool-use trajectories simply by varying the declared skeleton.

Engine forms. The runtime provides two direct implementations of Equation 5 and two compositional coordinators built over them. First, a train-side adapter invokes the model's forward generation pipeline directly in evaluation mode to fill the shells of a request in process. Second, dedicated serving adapters for SGLang, SGLang-Diffusion, vLLM, vLLM-Omni, and FastVideo translate ancestor primitives and stage sampling parameters into native serving requests and pack the outputs back into typed layer fields. At the compositional level, a multi-stage composed engine implements Equation 5 over a fixed multi-shell skeleton such as Equation 4 by invoking child engines sequentially across stages, whereas a multi-turn agentic harness composes Equation 5 dynamically across interaction turns—alternating `fork`, single-turn engine execution, and `observe` to return a trajectory with additional layers. Equation 5 thus standardizes the trajectory data boundary rather than imposing a single generation kernel.

3.5 Training folds the tree into per-stage updates

After rollout completes, the post-rollout pass folds feedback from the frontier back toward the root before executing per-stage parameter updates. First, the reward role scores the frontier layer—such as the image layer P_2^{diff} in Equation 4. Next, `propagate_rewards` reduces rewards from each scored child layer P_i onto its unscored parent layer P_{i-1} via mean, maximum, or sum, propagating downstream image feedback to the upstream AR rewrite layer P_1^{AR}. For group-relative objectives, `compute_advantages` then assigns credit on each trainable layer by grouping rows according to their ancestor ID at a specified depth—by default the immediate parent in P_{i-1}, or the root prompt in P_0—and normalizing rewards within each group. On the image layer of Equation 4, these two depths correspond to comparing images generated from the same rewrite versus comparing all images generated from the same root prompt; switching between them requires only a lineage depth parameter rather than an external table join (Section 8.3). Both reward reduction and group normalization rely on the sibling-contiguity invariant of Section 3.1 to process each sibling group as a contiguous slice. Group-normalized advantages are computed and attached on the driver before data-parallel scattering so that training shards receive precomputed credit without reconstructing sibling groups (whereas token-level GAE is computed per sequence on the training workers; Section 5).

During the update phase, the trainer routes each trainable layer P_i—matched by its stage sampling-parameter type—to a corresponding *training track* that pairs a stage algorithm with a `TrainStack`, while skipping frozen layers. In a decoupled two-stage workflow such as prompt enhancement, the AR rewrite model and the diffusion generator maintain separate training stacks and optimizer steps; freezing the rewrite stage simply disables the AR track so only the diffusion layer is replayed and updated. When both stages share a single backbone, as in HunyuanImage-3.0, a unified training stack replays both the AR and diffusion layers and back-propagates their combined losses through a single shared optimizer step.

3.6 Component interfaces for stage replay and optimization

Each training track separates stage optimization into three interacting abstractions: a trajectory-layer projection, a stage algorithm, and a training stack.

Trajectory-layer projection. For a trainable stage at depth i, layer P_i supplies the core tuple (C_i, Z_i, A_i) comprising model-facing conditions C_i, the stage replay segment Z_i (`TextSegment` or `LatentSegment`), and advantages A_i.

Stage algorithm. The stage algorithm encapsulates the stage-specific loss evaluation and backward pass over a single shard:

$$\mathcal{A}_{\text{stage}} : \ (C_i, Z_i, A_i; \theta) \longmapsto (\mathcal{L}(\theta), \nabla_\theta \mathcal{L}(\theta)). \tag{6}$$

The core objective signature consumes only (C_i, Z_i, A_i) and the active model parameters θ. When an objective requires auxiliary per-row state—such as metadata in M_i or token-level critic value estimates

stored in the segment—an optional per-shard preparation hook reads the enclosing `Part` P_i before micro-batching; algorithms that require a frozen reference policy or exponential moving average (EMA) teacher hold an explicit backend model handle.

Train stack. The `TrainStack` partitions shards into micro-batches, optionally caches an old-policy replay anchor under the training micro-batch layout prior to multi-epoch updates, orchestrates gradient accumulation and clipping, reduces global loss metrics across ranks, and steps the optimizer.

Under this division of responsibilities, AR objectives replay token log probabilities from `TextSegment`s and diffusion objectives replay selected denoising transitions from `LatentSegment`s, comparing the replayed likelihoods or transition statistics against those recorded during rollout, cached from a pre-update replay pass, or evaluated under a reference model. Supervised fine-tuning and teacher-distillation objectives ignore A_i, whereas the forward-process DiffusionNFT objective consumes group advantages A_i through the same (C_i, Z_i, A_i) interface.

Because model bundles and stage algorithms interact solely through typed layer fields, they compose without pairwise glue code. Each model family is integrated once as a bundle whose rollout adapters and trainable stages populate and replay typed segments (C_i, Z_i), and each algorithm is implemented once as an objective over those replayed segments. Any shared algorithm can therefore pair with any model stage that produces and replays the required segment type—AR objectives with `TextSegment` stages and diffusion objectives with `LatentSegment` stages—reducing the integration complexity of N_{mod} models and N_{alg} shared algorithms from $N_{\text{mod}} \times N_{\text{alg}}$ to $N_{\text{mod}} + N_{\text{alg}}$. When a model requires a custom loss, it introduces a stage-algorithm subclass without modifying existing model bundles or shared objectives; conversely, variants such as DanceGRPO and MixGRPO that retain the `FlowGRPO` loss (Liu et al., 2025) and alter only the SDE transition kernel or step schedule are expressed as configuration recipes rather than new algorithm classes.

We do not claim algorithmic novelty in the objectives themselves: our Group Relative Policy Optimization (GRPO) implementation follows the group-relative formulation of Shao et al. (2024), our Proximal Policy Optimization (PPO) implementation follows the clipped surrogate objective of Schulman et al. (2017), and our diffusion objectives directly implement prior diffusion-RL estimators (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025). Instead, the systems question is whether a single trajectory IR and execution substrate can support all of these objectives while faithfully preserving their distinct sufficient statistics.

4 From a Program to a Distributed Execution

Figure 2 separates the logical roles of the feedback program (top) and the trajectory IR that connects them (middle) from the runtime mechanisms and deployment topologies that execute them (bottom); within this loop, credit assignment runs either on the driver via group normalization or on the training workers via generalized advantage estimation (GAE).

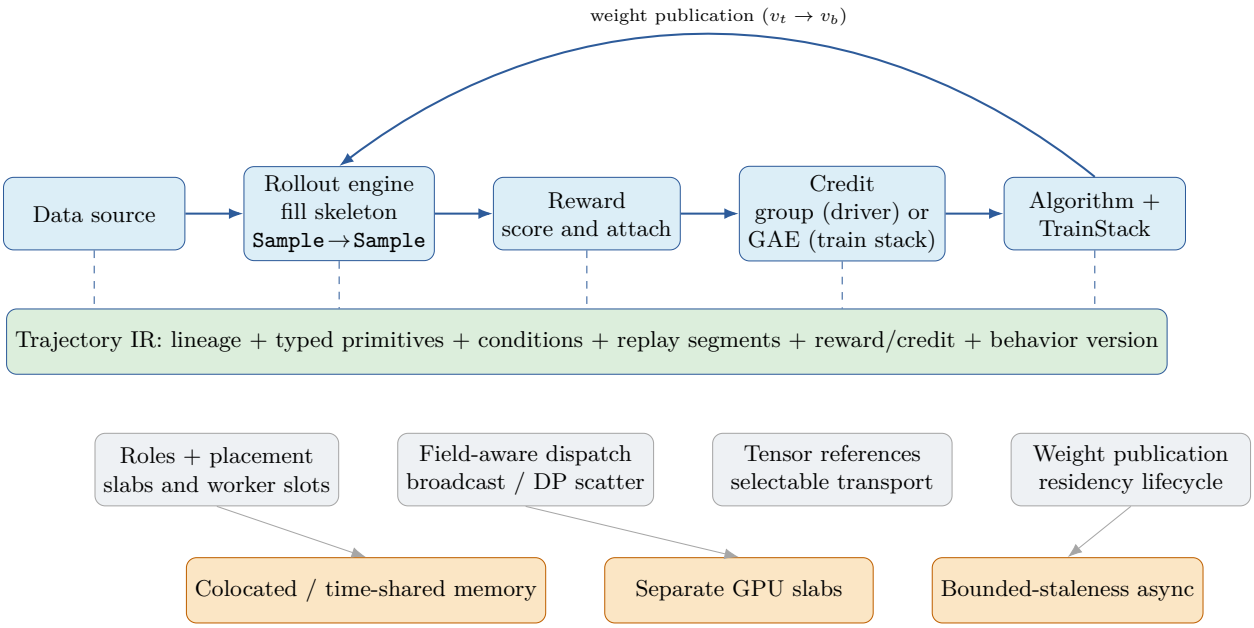


Figure 2: Source-derived UniRL architecture. The top row is the feedback program; the green bar is its stable data boundary. The lower rows map logical roles to communication and resource policies. Not every topology is valid for every engine; startup validation rejects unsupported combinations.

4.1 Logical roles, device slabs, and worker slots

A `DevicePool` creates one Ray worker per GPU initially and reserves one strict-packed placement group per node. Each worker occupies a slot on its GPU: slot 0 exists from startup, further slots are created on demand up to a per-GPU limit, and every slot reserves an equal fraction of that GPU. A top-level placement scope claims a fraction of the whole device pool from the still-unclaimed devices; a nested scope takes a fraction of its parent's devices. Roles instantiated within a scope either share the scope's existing worker slot on each device—enabling direct in-process sibling access and memory time-sharing—or allocate isolated slots on those same devices, which communicate through the per-GPU store transport. Disjoint top-level scopes form separate training, rollout, or reward slabs.

Each role is defined as a Python class derived from `Remote`. Instantiating a role inside a placement scope returns a driver-side handle that constructs one role replica on each selected worker and derives data-, tensor-, pipeline-, sequence-, and expert-parallel rank metadata from the role configuration. Method decorators declare which ranks execute and how the handle dispatches arguments and reassembles return values (broadcast to all target ranks, positional shard scatter, or root-group DP scatter using the field algebra of Section 3). The trainer is thus a single-controller program on the driver, while each dispatched method may run multi-controller distributed training or serving across the role's workers. This control split resembles HybridFlow's separation of orchestration from distributed computation (Sheng et al., 2024); UniRL differs by routing a single lineage-preserving, multimodal trajectory IR across heterogeneous generative stages.

4.2 Tensor references decouple structure from movement

In the single-controller design, every role method is a remote call issued by the driver, and its result returns to the driver before being dispatched to the next role. If trajectories carried dense tensors across these hops, latent snapshots, token log probabilities, and decoded media would pass through the driver on every call, and the driver would have to materialize them merely to split, select, or scatter complete root groups. UniRL therefore routes control through the driver but keeps tensor data on the workers. When a role method returns, each participating worker stores its dense tensor leaves in a local transport store and *dehydrates* each

tensor into a `TensorRef`: an ordered list of half-open row spans over opaque handles, together with shape, dtype, and device metadata. The driver receives only this dehydrated trajectory, so root-group selection and contiguous slicing construct new span views without moving tensor payloads. When a subsequent call targets a different role or shard layout, cross-worker transports localize each reference for its destination worker, leaving already-resident spans in place and transferring remote spans worker-to-worker. Each destination worker then *hydrates* all referenced spans in a single batch—zero-padding spans whose trailing dimensions differ to a common width—and writes the dense tensors back into the nested object in place before invoking the method. Driver-side control logic materializes only small scalar or vector fields, such as rewards, on demand.

The transport interface exposes four operations: store tensors, resolve handles, identify references, and (for cross-worker backends) localize references for destination workers. The audited source provides (i) an in-process colocated per-worker store, (ii) a per-GPU store using CUDA inter-process communication (IPC) and NVIDIA Collective Communications Library (NCCL) transfers, and (iii) a TransferQueue path backed by either built-in in-memory storage (`SimpleStorage`) or Mooncake-compatible distributed storage (Qin et al., 2024); we use Mooncake's storage interface purely as a transport backend rather than a claimed contribution.

The driver can therefore reorganize trajectory lineage and metadata without gathering dense GPU payloads onto the controller. Whether this indirection yields net memory and transfer savings relative to flat ID-joined rows is an empirical question under RQ3 (Section 8.3; the paired E5-R microbenchmark remains open) rather than an a priori assumption.

4.3 Residency lifecycles and weight publication

Training and generation often hold different copies or formats of a policy. We distinguish two version counters: the *optimizer version* v_t, which counts committed optimizer steps on the training stack, and the *behavior version* v_b, which the rollout engine stamps onto each generated frontier layer. In every deployment, a due weight publication completes before the generation it serves; how publication interleaves with memory residency depends on whether training and rollout share devices.

Colocated slabs. When training and rollout time-share a single GPU slab, the rollout engine is put to sleep—releasing its weight and KV-cache memory—between generations and woken before the next rollout, while training state may be offloaded to host memory while the engine is awake. All colocated schedules preserve two invariants (Appendix B): any due weight publication completes before generation begins, and the engine is put back to sleep before offloaded training state is restored for the training update. When no offload is needed during wake-up, the default schedule (used by the AR, agentic, and prompt-enhancement trainers) wakes the engine, publishes weights if due, offloads training state if enabled, and runs rollout, after which it puts the engine to sleep and restores the training state. When peak GPU memory cannot hold both the full training state and an awake engine, the AR and unified-model trainers use one of three adaptations (Table 17): (i) if no publication is due, offload training state before waking the engine; (ii) if the engine can allocate weight memory separately from KV cache, wake weight memory first, publish weights, offload training state (or offload optimizer state before waking weights and model shards after publishing for vLLM's native weight-transfer engine), and then allocate the KV cache; or (iii) when an AR engine is pinned to a single GPU of the slab or a unified-model trainer drives separate AR and diffusion engines, extract due weights first, offload training state, wake the engine(s), and publish the extracted weights. The diffusion trainers configure residency per role (training, rollout, reward): before waking the engine, they extract the adapter if training will be offloaded and offload every non-resident role; they then wake the engine, publish weights if due, put the engine to sleep after generation unless the rollout role is marked resident, and restore the offloaded training stack before its replay and optimizer update. A train-side rollout adapter reads the live training model directly, so training state is never offloaded for it.

Separate slabs. When training and rollout occupy disjoint GPU slabs, the two roles do not contend for device memory:

1. at startup, establish a one-time communication handshake between the training shards and rollout workers;
2. at each publication boundary, transfer updated parameters across slabs (subject to the quiescence discipline of Section 4.5 for asynchronous trainers);
3. generate trajectories and stamp the frontier layer with the engine's current behavior version v_b.

Rollout never offloads training state to free space for the engine. On its separate slabs, the synchronous diffusion trainer still puts the rollout engine to sleep between rollouts unless the rollout role is configured as resident, and offloads training state after generation only when a non-resident training role time-shares the training slab with a reward model. The asynchronous trainers keep the rollout engine awake throughout training, and the asynchronous diffusion trainer additionally requires a resident reward role.

Weight-publication handlers (`unirl/distributed/weight_sync/`) support both in-process and cross-slab Low-Rank Adaptation (LoRA) adapter transfer as well as full-parameter publication via serialized tensor payloads (`TensorWeightSync`), CUDA IPC (with handles exchanged over ZeroMQ or through vLLM's native weight-transfer engine), SGLang checkpoint-engine IPC, NCCL broadcast, and filesystem checkpoint handoff (`CheckpointWeightSync`). For multi-stage models, a composed rollout engine can also route per-track weight updates to the targeted child engine.

Version stamping depends on the trainer path. The asynchronous trainers (via the rollout manager; Section 4.5) and the synchronous agentic trainer set $v_b = v_t$ upon each weight publication, so v_b records exact optimizer-version provenance. The remaining synchronous trainers (AR, diffusion, prompt-enhancement, and unified-model) leave $v_b = 0$ across rollouts, except when configured with vLLM's native weight-transfer engine, which increments v_b by one on each received publication.

The source rejects combinations whose lifecycle has not been validated, such as a train-side rollout on a physically separate slab or selected EMA/offload combinations. This fail-closed policy is important because a partially awakened engine or silently stale adapter can produce plausible but invalid training data. Section 8 therefore includes replay-consistency and failure-injection checks alongside throughput.

4.4 Synchronous execution

The synchronous AR trainer performs rollout, reward, credit assignment, replay/update, optional evaluation, and checkpointing in order. The synchronous diffusion trainer can accumulate multiple rollout batches into a single optimizer window; spatially, it can colocate the reward role on the training slab, isolate it on a third GPU slab, or query an external reward service, and temporally, it can pipeline micro-batched reward scoring inside the rollout window to overlap generation with evaluation (Section 6). Across modalities, deployment topology is selected through the same placement-scope primitives. The synchronous AR trainer allocates a single colocated slab (optionally pinning the rollout engine to one GPU), whereas the synchronous diffusion trainer supports either a single colocated slab or separate training and rollout slabs. Disaggregated synchronous AR execution (`D0` in Section 8.4) is realized by running the asynchronous AR trainer (Section 4.5) with one prompt batch in flight, weight publication after every batch, and zero allowed policy lag ($\ell = 0$), so that rollout and training strictly alternate across separate slabs. In all cases, changing topology alters device ownership, residency scheduling, and weight publication without altering the trajectory IR boundary: the same `Sample` object flows from rollout through reward and credit assignment to the training stack. This invariance enables controlled deployment comparisons that hold the model, prompt stream, group size, reward function, generation and replay step or token budgets, and optimizer updates per rollout fixed while varying the slab layout, alongside the resulting changes in per-role DP width, residency handoff, and publication path (Section 8.3).

4.5 Bounded-staleness execution

Asynchrony changes correctness conditions, not just control flow. UniRL's asynchronous AR and diffusion trainers run on separate training and rollout slabs and share one manager. The driver submits whole prompt batches to the rollout manager only while they fit within the in-flight batch capacity, the freshness horizon

implied by the staleness allowance, and the batch count remaining before the next evaluation, checkpoint, or end-of-training boundary. A background dispatcher then feeds prompts to each rollout engine replica up to its configured request-concurrency limit. As sibling completions return, the manager assembles them by root ID; only complete root-prompt trees enter the ready buffer, and every generated layer must carry a behavior version v_b.

Let v_t denote the current optimizer version (counting committed optimizer steps) and u the configured number of optimizer updates per consumed rollout batch. The manager bounds policy lag in optimizer-update units:

$$\ell = v_t - v_b \leq ub, \tag{7}$$

where b is the staleness allowance in rollout-batch units (configurable for AR, and fixed to the publication interval minus one for diffusion). The lag filter assigns each root-complete tree the behavior version of its oldest generated layer and discards whole root groups with $\ell > ub$. Because a skipped non-finite gradient step does not advance v_t, an admitted group can span more than b elapsed rollout batches while still satisfying $\ell \leq ub$. When a weight publication is due at optimizer version v_t, the trainer and manager execute a quiescence barrier: the manager pauses new admission, finishes any root groups already in progress (or the prefix of groups needed to complete an unmixed single-version training batch), and discards any partially generated root groups that would otherwise straddle the version change while retaining queued root prompts that have not yet started generation (*pristine carry*). The trainer then publishes weights from the training stack to the rollout engine, the manager updates the engine's behavior version to v_t, and those unstarted root prompts are resubmitted under the new version. Evaluation and checkpointing require the manager's in-flight and ready buffers to be completely drained.

This bounded-staleness discipline differs from fully asynchronous optimization: weight publication remains a synchronous barrier, and no training batch may mix behavior versions. The asynchronous diffusion trainer admits at most one prompt batch in flight and requires each training batch to come from a single generation request, so the next rollout overlaps the training slab's replay and optimizer update phase; the asynchronous AR trainer can keep multiple batches in flight and dispatches replacement rollout requests to the manager before scoring rewards on the newly popped batch. Consequently, an empirical evaluation must report realized lag distributions, discarded work, and publication barrier time rather than relying on a nominal "async" label.

5 Instantiations of the Common Contracts

The following instantiations show which information crosses the common trajectory boundary and which computations remain stage-specialized. They illustrate the contracts at the design level, as implemented in the audited v0.2.0 source, rather than serving as an exhaustive inventory of supported models. Among these instantiations, the case studies (Sections 6 and 7) and main evaluation (Section 8) exercise single-stage AR post-training with group-relative objectives (GRPO in E0, E1, E3-AR, and E6; DRPO in E1), single-stage diffusion and flow post-training with Flow-GRPO (E0, E2, E3, E5-T, and the BAGEL, HunyuanImage-3.0, and MiniMax-H3 runs), and the cross-stage prompt-enhancement workflow with a frozen AR rewriter and trained diffusion stage (E4). By contrast, critic-based token-level GAE, the forward-process DiffusionNFT objective, joint training of the upstream rewrite stage, and multi-turn tool-interacting agentic workflows are implemented with stock recipes in the audited source but have no reported training runs in Section 8.

5.1 Autoregressive reasoning

An input layer P_0 carries root prompt text and metadata, and forking by the group size k appends a generated shell P_1^{AR} with parent-contiguous sibling IDs. An AR rollout engine fills each row's decoded text and packed `TextSegment`, after which the reward role attaches scalar or multi-component rewards. Group-normalized advantages are computed on the driver before the batch is scattered to training workers; when critic-based token-level GAE is used instead, advantages and returns are computed independently per generated sequence on the training workers after the train stack freezes the critic anchor. On each training shard, the train stack plans token-budgeted micro-batches, replays the AR stage on each `TextSegment`, and invokes the stage

algorithm to evaluate the policy loss and backward pass. Before dispatching the training layer across DP shards, the trainer may optionally apply token-length shard balancing, which permutes individual generated rows after driver-side group credit (and before per-sequence GAE, which is invariant to row order). This permutation preserves each row's identifier and precomputed advantage but intentionally breaks sibling contiguity, so no group-level reduction or normalization may follow it; in our evaluation, both the E1 GRPO recipe and E3-AR enable shard balancing, whereas the E1 DRPO recipe disables it (Section 8.1).

5.2 Diffusion and flow generation

To construct a diffusion request, the trainer forks the root prompt layer and records the stage sampling parameters, selected SDE transition indices, and initial-noise specification on the generated shell; the rollout engine then pins the σ-schedule onto the shell before denoising. Generation populates decoded media together with a `LatentSegment` whose stored denoising states and schedule metadata supply the sufficient statistics for the chosen stage algorithm. Reverse-process objectives such as Flow-GRPO replay the recorded SDE transitions and compare current transition log probabilities against a frozen rollout or pre-update replay anchor (Liu et al., 2025). Forward-process objectives such as DiffusionNFT instead operate on terminal clean latents rather than reverse-transition likelihood ratios (Zheng et al., 2025). Both families share the same train stack for micro-batching, gradient accumulation, and optimizer state, while the stage algorithm encapsulates their incompatible loss estimators.

5.3 Cross-stage and environment-interacting trajectories

In a prompt-enhancement workflow, the AR stage expands each root prompt into k_1 text rewrites and the diffusion stage expands each rewrite into k_2 images (Equation 4). The reward role scores the leaf image layer P_2^{diff}; when the rewrite stage is trained, `propagate_rewards` reduces each sibling block onto its parent rewrite in P_1^{AR}. In our E4 study (Section 8.3), the rewrite stage is held frozen and lineage is used to select the normalization scope for diffusion group advantages—grouping either the k_2 images under each rewrite or all $k_1 k_2$ images descended from the same root prompt. Preserving this ancestry across rollout scheduling and DP sharding avoids reconstructing prompt–rewrite–image correspondence via out-of-band joins; E4 verifies that both grouping scopes match an explicit ID-joined reference on synthetic fixtures and on recorded rollouts from the composed run, and then compares the two scopes across three-seed training runs (where both improve over the base model and their quality difference is inconclusive).

In a multi-turn tool-interacting workflow, each rollout episode forms a single-branch chain in which the policy emits an action-like text layer, the environment appends an observation layer via `observe`, and interaction repeats until termination or suspension. Because episodes sampled from the same prompt can terminate after different numbers of turns (yielding different layer depths L), the agentic trainer maintains one branch-one `Sample` per episode and forms a prompt group from repeated copies of the root prompt rather than siblings inside a single `Sample`. It then computes group-normalized advantages across the episodes in each group and flattens their generated turns for training, with every turn inheriting its episode's advantage; here, trajectory lineage serves to condition each turn on its full interaction history rather than to define intra-`Sample` sibling groups.

Before turning to the main E0–E6 evaluation program in Section 8, Sections 6 and 7 present two targeted case studies that examine how the reward-serving and rollout-engine contracts behave on instruction-guided image editing.

6 Case Study I: Reward Serving and Generation–Reward Overlap

Reward evaluation is the one role in the feedback loop whose computational cost is independent of the policy architecture. A lightweight contrastive preference model scores a batch in well under a second, whereas a vision–language judge with tens of billions of parameters can take as long as the generation it evaluates, requires a separate software environment from the trainer, and may not fit alongside the policy on the same accelerator. UniRL therefore fixes the reward interface at the trajectory boundary while treating placement, environment isolation, and scheduling relative to generation as orthogonal execution choices.

Source scope. This case study examines UniRL commit `651490e` (2026-09-30), which postdates the v0.2.0 design-audit commit analyzed in Sections 3–5, together with the shared-GPU mechanism proposed in UniRL pull request #492 (commit `8bb4119`). The measurements in Section 6.4 were collected as a standalone systems case study outside the E0–E6 evaluation program; their per-rollout records and table generators are archived in `artifacts/reported_results/2026-09-30-reward-service/`.

6.1 Scoring Contract and Failure Semantics

The trainer invokes a single method, `score_and_attach`, on the reward role. Like any data-parallel role method (Section 4), the call scatters the trajectory batch (`Sample`) across workers as root-complete shards; on each worker, the reward role constructs a typed request from its shard, queries a configured backend for scores, and returns an updated shard whose frontier layer P_L carries a per-row scalar reward and named component rewards in R_L. The request packages the frontier layer's decoded primitives X_L, non-text primitives from ancestor layers (such as the source image from P_0 in an image-editing task), both the original root prompt and any rewritten generation prompt, and the sample and group identifiers. Each backend specifies which prompt field it evaluates. When a prompt-rewrite stage precedes generation, the two prompt fields differ, and this setting determines whether the reward measures fidelity to the original user request or to the rewritten prompt.

To prevent silent propagation of invalid credit, the reward role enforces fail-closed preconditions and output checks around scoring. Before evaluation, it rejects any frontier layer that already carries rewards; for audio–video generation, startup validation also rejects training reward configurations that assign zero weight to prompt-to-video alignment, because optimizing audio–video agreement alone permits degenerate solutions that collapse both streams (such configurations remain permitted in evaluation-only mode). During evaluation, free-length autoregressive completions that reach the token limit without terminating receive either zero reward or a configured length penalty, and any row-level scoring failure—including any non-finite score returned by a remote backend—raises an exception rather than substituting a default zero. Because group-normalized advantage estimation couples every sibling under a prompt, a single corrupted reward would contaminate the entire group; failing closed prevents corrupted updates from entering the optimizer.

Table 2 summarizes the scorers that implement this contract: 19 in-process backends and 13 scorers available in the standalone reward service's registry. These span contrastive and preference models such as CLIP, PickScore, HPS, and ImageReward (Radford et al., 2021; Kirstain et al., 2023; Wu et al., 2023; Ma et al., 2025b; Xu et al., 2023), vision–language judges for composition, rendered text, and instruction-guided editing (Kamath et al., 2025; Wu et al., 2025; Luo et al., 2025), video and audio–video alignment models, and text verifiers for reasoning tasks. A remote backend can query a panel of scorers for each row and aggregate the results via a weighted sum, mean, minimum, or maximum while preserving individual component scores for logging. Inclusion in Table 2 indicates that the adapter code path exists in the repository; it does not imply that every listed scorer has been benchmarked in end-to-end training.

Table 2: Reward implementations at UniRL commit `651490e`, grouped by scored output modality. Names denote registry keys. *Present in the registry but disabled in the default service configuration.

Output scored	In-process backends	Reward-service scorers
Image	`clip`, `pickscore`, `hpsv2`, `hpsv3`, `hpsv3pp`, `image_reward`, `ocr`, `geneval2`	`clip`, `pickscore`, `imagereward`, `hpsv2`, `hpsv3`, `unified_reward`, `geneval2`, `geneval`*, `ocr`, `wise`
Image editing	–	`editreward`, `editscore`
Video	`videopickscore`, `videoclipdelta`, `videoalign`, `video`	`videoalign`*
Audio	`clap`	–
Audio–video	`imagebind`, `t2av_composite`	–
Text	`math_verify`, `mc_exact_match`, `llm_judge`	–
Routing	`per_domain`	–

6.2 Scorer Placement and Isolation Modes

The same scoring contract can be realized across four placement and isolation modes that trade off communication overhead against device-memory and dependency isolation.

In-process on colocated policy workers. An in-process backend loads the scorer directly inside the worker slot that hosts the training and rollout roles. Its device memory is governed by the per-role residency policy described in Section 4.3: each role (training, rollout, and reward) is configured as either resident or offloaded when idle, and the residency planner issues only the state transitions required between phases. When reward evaluation immediately follows generation, the reward role executes while the training state remains offloaded on host memory, avoiding redundant host–device transfers.

On a dedicated reward slab. A placement scope can allocate a fraction of the device pool as a separate GPU slab dedicated to the reward role. The driver scatters scoring calls across the workers of that slab, which hydrate the frontier layer's decoded media from `TensorRef` handles via worker-to-worker transport (Section 4), isolating heavier scorers from policy memory pressure while keeping the workflow within a single distributed program.

In a managed child process. Scorers that depend on dedicated serving engines or conflicting library versions often cannot be imported into the trainer's Python environment. A managed child backend spawns one subprocess per reward worker using a separate Python interpreter, binds it to a local loopback HTTP socket and that worker's assigned GPU, and communicates over a strict request–response protocol. Requests are transmitted in bounded chunks independent of the data-parallel shard size. Every item carries its sample ID, group ID, source rank, behavior version, and scorer version, all of which the child process must echo in its response. An idempotency key derived from these identifiers and a payload fingerprint allows retried requests to return cached scores without re-evaluating the model. The child process serializes scoring requests with its memory-lifecycle transitions and rejects non-finite outputs. The parent worker can either keep the child's model resident on the GPU or wake it only for the duration of each scoring call, supporting both text-to-image and image-editing requests in the audited implementation. Because the subprocess runs in an isolated interpreter, it can host a full vLLM serving engine (Kwon et al., 2023). For the vLLM-backed EditScore judge integrated in UniRL pull request #348, offloading the child engine to host memory takes 5.1 s (reducing its GPU footprint from 47 GB to 1.0 GB), waking it takes 0.3 s, and the returned scores are bit-identical across offload–wake cycles.

In a standalone service. A standalone reward service places an HTTP gateway in front of Ray actor groups (Moritz et al., 2018). Each configured scorer runs as a pool of replica actors that reserve dedicated GPUs and initialize their own package dependencies on startup. The gateway decodes each incoming batch once, groups items by requested scorer, dispatches requests across replicas in round-robin order, and awaits each scorer under an independent timeout. Scorer failures are reported per item and per scorer alongside successful results so that a single failed backend does not abort the entire batch; the trainer client treats any missing score as a row failure. Because actor groups within the Ray cluster share the base Python interpreter and PyTorch runtime, conflicts at the interpreter level require either the managed child backend or an external single-scorer HTTP server launched on a separate node, which is the deployment used for the remote judges in Section 6.4.

6.3 Overlapping Reward Evaluation with Generation

In the default synchronous diffusion loop, all data-parallel ranks finish generating their full shards before the driver dispatches reward scoring. When scoring uses a remote judge, this barrier introduces two sources of latency: the training GPUs remain idle while the remote judge processes the batch, and requests from all data-parallel ranks arrive at the judge simultaneously, creating a burst queue.

An optional worker-side wrapper, `RewardStack`, pipelines generation and scoring at the micro-batch level within the colocated rollout window without altering the outer trajectory contract. Running in the same worker slot as the rollout engine and the reward role, `RewardStack` partitions its shard's unfilled frontier shell

into micro-batches of a configured row count and invokes generation followed by reward evaluation on each micro-batch. The scored micro-batches are concatenated in their original row order so that sibling contiguity and downstream group credit assignment remain unchanged, allowing the driver to bypass its separate post-generation reward call. A micro-batch may be as small as a single row; if the underlying rollout adapter requires complete sibling groups per engine call, `RewardStack` rounds the micro-batch size up to the group boundary (the sibling-packing change in Section 7.3 removes this constraint for the vLLM-Omni BAGEL adapter). When `overlap` is enabled, `RewardStack` submits each completed micro-batch's scoring request in order to a dedicated background thread on the worker so that generation of the next micro-batch begins immediately; any scoring exception is surfaced once the in-flight generation micro-batch completes.

`RewardStack` enforces fail-closed startup validation on unsupported topologies and RNG states. Because it requires the rollout engine and reward role to execute within the same single-rank worker slot, startup validation rejects configurations with a separate rollout slab, a separate reward slab, a reward role that is offloaded during generation, or sequence- or tensor-parallel rollout groups in which multiple ranks collaborate on a single shard. Overlapped execution also imposes a reproducibility constraint: if an in-process scorer draws from the global PyTorch random number generator, background scoring threads would interleave with the diffusion sampler's noise draws and render rollouts non-deterministic, so such scorers are restricted to the serial micro-batch schedule. (By contrast, disaggregated asynchronous trainers overlap generation and scoring at the batch level: the asynchronous AR trainer refills generation slots before scoring completed batches, and an open UniRL pull request extends the asynchronous diffusion manager to release rollout capacity while remote reward requests are in flight.)

6.4 Empirical Evaluation of Micro-Batched and Overlapped Schedules

Experimental setup. We evaluate the three scheduling modes on a single node of eight NVIDIA H20 GPUs while fine-tuning BAGEL (Deng et al., 2025) with Low-Rank Adaptation (LoRA) for instruction-guided image editing, using eight data-parallel vLLM-Omni rollout engines (TP=1). Each rollout processes 32 root prompts with eight image edits per prompt (256 generated rows in total, or 32 rows per rank), which a micro-batch size of eight partitions into four micro-batches per rank. On a second node, an 8B or 72B EditScore vision–language judge (Luo et al., 2025)—served by a single tensor-parallel vLLM instance behind the standalone single-scorer HTTP server—scores each source–edit image pair. Each run executes two to three rollouts; we exclude the first rollout as startup warm-up and pool steady-state rollouts across repeated sessions on two allocations of identical hardware. Phase durations are recorded by the driver's monotonic phase timers.

Because these runs were collected prior to the conditioning-injection fix analyzed in Section 7.1, the image-editing wall-clock durations in Table 3 and Figure 3 are rescaled estimates reflecting the restored conditioning path. Specifically, generation durations are scaled by the ratio between the fixed and unfixed conditioning paths measured at 16 rows per rank in Table 7, under the assumption that the same factor applies at 32 rows per rank across all micro-batches. Reward-side durations—judge service time, queueing delay, and non-overlapped reward wait time—are retained directly from the measurements, as the judge evaluates the same number of 512×512 image pairs at identical request batch sizes. When the judge has idle capacity during generation, the full generation time reduction carries over to end-to-end rollout time; when the judge is saturated, only the earlier arrival of the first micro-batch reduces total wall-clock time. Both the raw measurements and the rescaling model are archived in the artifact bundle. Because the rollout counts are small and cover a single model and workload shape, these results characterize the mechanism's behavior and approximate magnitude rather than a multi-seed performance distribution. In 13 of the runs, worker-side tracing additionally logged the start and completion timestamps of every generated micro-batch and HTTP reward call, while the judge server logged request arrival and completion times. Judge service times, total judge busy time, queueing delays, and per-pair scoring costs reported below are taken directly from these traces; arrival timestamps and overlap windows that depend on generation speed follow the rescaled estimate and are explicitly labeled as estimated.

Table 3: Generation plus reward wall-clock time per rollout across three scheduling modes (mean and min–max range over steady-state rollouts). The upper two blocks pair BAGEL image editing with a remote EditScore judge; the bottom block pairs BAGEL text-to-image generation with an in-process PickScore backend. The overlapped entries pool the merged implementation (which queues all completed micro-batches) with an earlier prototype (which kept at most one request in flight); the artifact bundle reports both variants separately. †Estimated for the restored conditioning path of Section 7.1 from measured reward-side traces; rollout counts reflect the underlying traced runs.

Schedule	Rollouts	Generate + reward (s)	Range (s)	vs. whole shard
EditScore 8B, remote node				
Whole shard, driver-dispatched reward†	4	102.0	101.1–103.0	–
Micro-batches, serial†	5	86.9	86.3–87.3	−14.8%
Micro-batches, overlapped†	7	82.3	82.0–82.7	−19.4%
EditScore 72B, remote node				
Whole shard, driver-dispatched reward†	2	155.0	154.5–155.4	–
Micro-batches, serial†	4	138.8	138.2–139.8	−10.4%
Micro-batches, overlapped†	6	140.1	138.1–142.9	−9.6%
PickScore, in-process				
Whole shard, driver-dispatched reward	2	56.0	56.0–56.0	–
Micro-batches, serial	2	52.4	52.4–52.5	−6.3%
Micro-batches, overlapped	2	52.3	52.3–52.3	−6.5%

Regime 1: Judge with spare capacity. With the 8B EditScore judge, serial micro-batching reduces combined generation and reward time from 102.0 s to 86.9 s (14.8%), and overlapping generation with scoring further reduces it to 82.3 s (19.4%). Generation itself requires approximately 73 s across all three schedules (Figure 3); the schedules differ in how much reward latency remains exposed after generation completes. Under the whole-shard baseline, all eight ranks submit their 32-pair requests simultaneously at the end of generation; the server processes them sequentially at 3.08 s per request, so the last rank waits 24.6 s for a score that takes the judge 3.08 s to compute. Under serial micro-batching, the eight ranks contend simultaneously only on their first micro-batch, where the maximum wait is 9.6 s. Those staggered completion times desynchronize the ranks on subsequent micro-batches, allowing later requests to reach an idle judge and return in a median of 1.3 s. Under overlapped micro-batching, generation proceeds without waiting for reward responses, so the ranks remain synchronized and contend on every micro-batch; however, those queueing delays occur concurrently with the generation of subsequent micro-batches, leaving only the final micro-batch's scoring latency exposed after generation finishes. Total judge busy time is 37.7 s per rollout, which fits within the estimated 54.8 s window between the arrival of the first micro-batch and the completion of generation.

Regime 2: Saturated judge. With the 72B EditScore judge, serial micro-batching reduces combined generation and reward time from 155.0 s to 138.8 s (10.4%), while overlapped micro-batching yields 140.1 s (range 138.1–142.9 s versus 138.2–139.8 s for serial), showing no further improvement. In both micro-batched schedules, the 72B judge remains continuously busy from the arrival of the first micro-batch (an estimated 18.2 s into the rollout) until the final request completes. Consequently, total rollout plus reward time is lower-bounded by the first request's arrival time plus the judge's total service work, regardless of whether subsequent requests are issued serially or overlapped. Total judge service work per traced rollout ranged from 119.7 to 121.4 s under the serial schedule and from 119.5 to 124.5 s under the overlapped schedule, with rollout-to-rollout variation of up to 3 s within the same arm; the small difference between the serial and overlapped means in Table 3 is within this run-to-run variation. Once the judge is fully utilized throughout the overlap window, further wall-clock reductions require either additional judge replicas or lower per-request scoring cost.

Conditions for judge saturation. Whether a deployment enters the saturated regime depends on the ratio of total reward service work to the generation overlap window rather than on judge parameter count

alone. Pairing a 72B judge with a 7B-active-parameter policy and serving all eight data-parallel worker ranks from a single judge replica intentionally stresses the reward bottleneck. Even at the lower per-pair cost of whole-shard 32-pair requests, a single 72B judge replica requires 76.9 s of service time per rollout, which already exceeds the estimated 54.7 s window between the arrival of the first micro-batch and the end of generation; smaller eight-pair micro-batches further increase total judge work to 120.4 s (analyzed below). Operating the 72B judge with spare capacity at this rollout shape would therefore require multiple judge replicas, while cross-request continuous batching or prefill–decode separation would reduce the replica count needed. We include the single-replica 72B configuration as a boundary stress test of the overlap schedule rather than a recommended production provisioning.

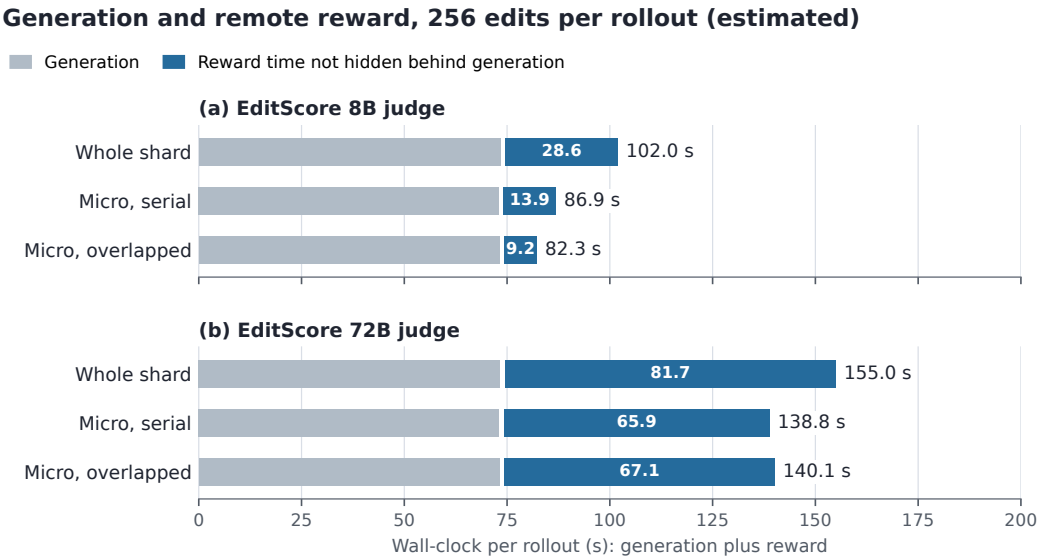


Figure 3: Breakdown of rollout wall-clock time for the two remote EditScore judges in Table 3 (estimated as described in the table caption). Generation duration is constant across schedules; the schedules differ in the portion of reward evaluation time exposed outside generation.

Batch-size overhead of micro-batched requests. Because the standalone single-scorer server processes one HTTP request at a time without cross-request batching, splitting a 32-pair shard into four eight-pair requests trades per-request batching efficiency for an earlier start. On the 72B judge, an eight-pair request takes 3.76 s (0.47 s per pair) whereas a 32-pair request takes 9.61 s (0.30 s per pair), raising total judge work per rollout from 76.9 s to 120.4 s; on the 8B judge, per-pair cost rises from 0.096 s to 0.150 s, raising total judge work from 24.6 s to 38.5 s. Micro-batched scheduling reduces end-to-end wall-clock time despite this overhead because judge evaluation begins one quarter of the way through generation rather than after the full shard finishes. When requests are split into smaller chunks *without* interleaving them with generation, only the batch-size penalty remains: in a post-generation ablation at 16 rows per rank, splitting the whole-shard call into two consecutive eight-pair requests increased combined generation and reward time from 52.2 s to 57.2 s on the 8B judge and from 86.1 s to 100.7 s on the 72B judge. Enabling cross-request continuous batching on the judge server would eliminate this small-batch compute penalty while preserving the early-start advantage of micro-batching.

In-process scoring baseline. When PickScore runs in-process on the policy workers, reward evaluation accounts for roughly 1 s of compute per rollout, and `RewardStack` reduces combined generation and reward time from 56.0 s to 52.4 s primarily by eliminating the separate driver-level dispatch hop. Because in-process scoring is already negligible relative to generation, background-thread overlap provides no additional reduction over serial micro-batching (52.3 s overlapped versus 52.4 s serial).

Interaction with training-phase memory collection. Executing multiple generation and scoring micro-batches within a single rollout leaves uncollected Python objects and cached allocations in the worker heap. When the subsequent training phase began without an explicit garbage-collection and cache-flush pass at the rollout boundary, training step time increased from 178.1 s to 192.2 s in the 8B-judge runs and from 54.5 s to 58.0 s in the in-process PickScore runs. The merged `RewardStack` implementation therefore triggers an explicit garbage-collection and CUDA cache-flush pass at the end of each rollout before returning the scored shard. Because the training role reconstructs its own model conditions C_i during replay independently of the rollout engine's conditioning path (Section 7.1), these training-phase durations are unaffected by the rollout conditioning bug and are reported as direct measurements rather than rescaled estimates, illustrating why rollout scheduling changes must be evaluated across the full post-training iteration.

6.5 GPU Sharing Across Lightweight Scorers via MPS

By default, the standalone reward service allocates whole GPUs to each scorer actor. While this provides hardware fault isolation, a panel of lightweight preference models leaves a high-memory accelerator largely underutilized. Although Ray supports fractional GPU allocation to colocate multiple actors on a single device, standard CUDA contexts from separate processes are time-sliced by the driver. NVIDIA Multi-Process Service (MPS) allows kernels from distinct processes to execute concurrently on the same GPU (NVIDIA Corporation), preserving per-scorer process and dependency isolation while overlapping their compute kernels.

UniRL pull request #492 introduces an opt-in MPS runtime mode for the standalone reward service. The service can either manage the node's MPS control daemon directly—starting it before any scorer actor initializes CUDA and stopping it after all actors exit—or attach to an externally managed daemon. Each scorer configuration may specify an active-thread percentage and a device-memory allocation cap, which are exported to the actor's environment prior to CUDA initialization. Startup validation rejects MPS configurations that combine device sharing with tensor-parallel scorers, unqualified scorer backends, or half-precision weights running below a 100% active-thread cap. During shutdown, the service halts request admission, drains in-flight GPU work, and terminates actors before stopping the MPS daemon. Because the MPS memory cap bounds per-client allocations without providing hardware memory protection, all actors sharing a GPU belong to a single fault domain, which is recorded in the service's health metadata.

Table 4: Throughput gain (%) of NVIDIA MPS over fractional GPU colocation without MPS for two or three replicas of a single scorer on one NVIDIA H20 GPU, as reported in UniRL pull request #492; B_s denotes the scorer batch size. Dashes indicate batch sizes at which the scorer implementation does not execute a single batched forward pass.

	Two copies			Three copies		
Scorer	B_s=1	B_s=4	B_s=8	B_s=1	B_s=4	B_s=8
CLIP	+11.2	+20.0	+14.8	+16.5	+24.4	+19.2
HPSv2	+17.4	–	–	+48.4	–	–
ImageReward	+24.0	–	–	+40.8	–	–
HPSv3	+20.6	+11.2	–	+22.6	+12.8	–
GOT-OCR	−2.8	–	–	−3.2	–	–

Reported qualification results. Table 4 summarizes the qualification benchmarks reported in UniRL pull request #492 on NVIDIA H20 GPUs (these measurements were not independently rerun for this report). For heterogeneous panels colocated on a single H20 GPU, the pull request reports a median throughput gain of 24.9% for three lightweight scorers and 9.6% for two, smaller gains when a heavier vision–language scorer dominates device time, and no gain for an autoregressive OCR scorer. On the deterministic test fixture, output scores were bit-identical with and without MPS enabled. Profiling in the pull request indicates that a single CLIP replica at B_s=1 utilizes roughly half of the device's execution time, allowing MPS to fill the inter-kernel gaps with work from colocated replicas; once a single scorer saturates the GPU, MPS provides no additional throughput.

Numerical correctness boundary. The same qualification identified a numerical defect on the tested H20 driver stack when restricting the MPS active-thread percentage below 100%: half-precision matrix multiplications produced non-finite outputs in CLIP and finite but numerically incorrect values in PickScore. Consequently, the runtime rejects half-precision scorers configured with less than a 100% active-thread share. A separate review on a different GPU model and driver version did not reproduce the numerical corruption and observed negligible colocation overhead even without MPS because the workload was bottlenecked by CPU preprocessing and HTTP serialization. Both the throughput benefit and the sub-100% thread-cap hazard are therefore hardware- and driver-dependent. Only CLIP and PickScore at a 100% active-thread share are marked qualified in UniRL pull request #492; vision–language judges, long-horizon stability runs, and sharing GPUs between reward actors and policy workers remain outside its qualified scope.

7 Case Study II: Rollout Fidelity and Request Shaping

Whereas Section 6 examined how generated trajectories are scored, this case study examines the boundary between the trajectory IR and an external rollout engine. Even when a rollout adapter returns a structurally well-formed `Sample`, three engine-boundary concerns govern whether optimization updates match the scored behavior policy and how efficiently rollouts execute: first, the rollout engine and the training role must condition on identical multimodal context representations (Section 7.1); second, the adapter must propagate per-prompt output canvas resolutions while keeping trajectory segments dense (Section 7.2) and balancing variable-resolution work across data-parallel ranks (Section 7.5); and third, engine requests should pack contiguous sibling rows under a shared prompt so that source conditioning is encoded once per group rather than once per sibling row (Section 7.3). Verifying and optimizing these properties requires explicit cross-boundary invariants and request-shaping contracts.

Source scope. This case study examines four stacked changes to the vLLM-Omni BAGEL rollout path proposed in UniRL pull requests #547–#550 (open at the time of writing), affecting both text-to-image (`t2i`) and instruction-guided image-to-image editing (`it2i`) workloads. All measurements below were collected on a single node of eight NVIDIA H20 GPUs, with per-run records and table generators archived under `artifacts/reported_results/2026-10-04-rollout-fidelity/`. The dynamic load-balancing prototype evaluated in Section 7.5 was measured on an experimental branch without an open pull request.

7.1 Conditioning Mismatch Undetected by the Importance Ratio

In instruction-guided image editing (`it2i`), the policy conditions on a source image alongside the edit instruction. To guarantee rollout–replay consistency, UniRL's rollout adapter constructs this conditioning using the model bundle's image transforms and vision encoder, injecting the resulting key–value contexts directly into the vLLM-Omni engine so that the training role replays against identical contexts during optimization. Following an upstream serving-engine migration, however, three helper functions in the BAGEL pipeline continued to read the injected conditioning field from the outer request-batch wrapper after the caller had already unwrapped it into an individual request object. Because the looked-up attribute was absent on the unwrapped request, the engine silently ignored the injected contexts and fell back to its internal image-to-image preprocessing path—applying a different image transform at a fixed resolution, drawing a sampled rather than posterior-mean VAE latent encoding, and sizing the output canvas to the source image rather than the recipe target.

Despite this bug, training completed without raising an error: every image-editing rollout generated samples under the engine's fallback conditioning (5,938 context tokens) and replayed them under the trainer's conditioning (2,263 context tokens at the same 512×512 source resolution). Notably, standard importance-ratio monitoring did not flag the discrepancy. Before the first optimizer update of a rollout, the current policy θ equals the behavior policy π_v, so the importance ratio ρ against the rollout behavior anchor should equal 1 up to floating-point noise. In practice, the observed ratio remained close to 1 despite the mismatch: the maximum first-update deviation $|\rho - 1|$ was 5.0×10^{-5} with the conditioning bug versus 3.4×10^{-5} after the fix, well within the 5.4×10^{-5} deviation observed in healthy image-editing runs (Table 5). This insensitivity arises from how denoising transition log probabilities are reduced. Both the rollout engine and the trainer

compute the Gaussian log-density of each denoising transition by averaging over all latent elements rather than summing across them; consequently, a perturbation to the predicted velocity mean—even one large enough to alter the generated image—is divided by the number of latent dimensions (tens of thousands of elements at 512×512 resolution) before entering the log-probability ratio. Under element-mean reduction, the importance ratio cannot distinguish a structural conditioning mismatch from benign numerical noise. Moreover, when a recipe uses the trainer-recomputed replay anchor (the default in several diffusion configurations), the baseline log probabilities are recomputed on the trainer itself, making the first-update ratio identically 1 by construction.

To detect such discrepancies directly, UniRL adds a structural token-layout check across the rollout–replay boundary (UniRL pull request #548). The engine hook that injects initial noise now also records the conditioning layout and output canvas geometry—specifically, the generation-context token length, its positional offset, and the rendered height and width—which return through the capture channel into the frontier layer's model conditions C_i and are compared row-by-row against the context reconstructed by the training role during replay. Any mismatch raises an exception on the first replay pass and logs both layouts. Because the training role already constructs the replay context, the check adds no tensor-compute overhead; conditioning paths that pass pre-materialized context tensors directly from rollout to replay bypass the check. Table 5 summarizes the fault-injection and regression probes verified by this check. The importance-ratio check is retained as an optional secondary diagnostic for detecting weight-publication desynchronization or mismatched LoRA adapters, where the model parameters rather than the conditioning layout differ between rollout and replay.

Table 5: Validation probes for the structural token-layout check across text-to-image (`t2i`) and image-to-image editing (`it2i`) paths. "Engine" and "Trainer" columns report the generation-context token counts on each side. The final column reports the maximum first-update importance-ratio deviation $|\rho - 1|$ in the same run.

Probe	Path	Engine	Trainer	Layout check	$\lvert\rho - 1\rvert$
Main before the injection fix	it2i	5,938	2,263	raises at the first replay	5.0×10^{-5}
Trainer prompt lengthened by three tokens	t2i	63	66	raises at the first replay	–
Trainer source image halved in width	it2i	2,268	1,161	raises at the first replay	–
After the injection fix	it2i	2,263	2,263	passes	3.4×10^{-5}
Healthy run (rollout anchor)	t2i	equal	equal	passes	8.6×10^{-6}
Healthy run (rollout anchor)	it2i	equal	equal	passes	5.4×10^{-5}
Healthy run (replay anchor)	it2i	equal	equal	passes	0

Restoring the injected conditioning (UniRL pull request #547) affected both rollout latency and initial judge scores. Generation time for 16 edits per rank decreased from 49.2 s to 36.8 s (25%), because the engine's fallback path encoded the source image into nearly four times as many vision tokens as UniRL's encoder path (5,938 versus 2,263 total context tokens). Simultaneously, the mean EditScore reward on the first rollout—evaluated on identical prompts prior to any optimizer update—increased from 2.80 to 4.62 (Figure 4), reflecting the quality difference between the two conditioning representations on the pretrained checkpoint.

7.2 Per-Group Output Canvas Resolution

Previously, every rollout batch required a single uniform output resolution fixed by the recipe, because a layer's `LatentSegment` Z_i stores latents as a single dense tensor and the rollout adapter passed one canvas size per batch to the engine. Production prompt datasets, however, contain examples with diverse target aspect ratios and resolutions; forcing a uniform canvas either wastes compute on low-resolution examples or downsamples and distorts high-resolution ones.

UniRL pull request #549 allows each root example to specify its target canvas dimensions in the root metadata M_0. During rollout, the adapter resolves each frontier row's target canvas from its root ancestor in P_0 (falling back to the recipe default) and dispatches one engine call per contiguous run of rows that share

a canvas size. To keep the latent segment tensor dense across a mixed-resolution batch, smaller canvases are right-padded along the token dimension to match the recipe's maximum canvas size, and the trainer slices each row back to its true token count during replay; canvases exceeding the recipe maximum are rejected at admission. Because all sibling samples forked from a root prompt share the same canvas, group-relative credit assignment, the micro-batched `RewardStack` of Section 6.3, and downstream scorers require no modification. Datasets without per-example canvas metadata execute identically to the fixed-resolution path. Rollout backends that support only a single resolution per batch—including the train-side diffusion adapter, non-BAGEL serving adapters, and video pipelines—explicitly reject the per-row canvas metadata field rather than silently rendering at the default resolution.

This padding scheme applies to objectives whose replay anchor stores a single scalar per denoising transition (such as Flow-GRPO). Objectives that store per-token replay anchors or consume the dense latent field directly still require a uniform canvas across the batch and now fail fast with a descriptive error when given mixed-canvas inputs.

Table 6: Generation time (seconds) for one prompt group of four sibling samples across three output canvas resolutions on a single H20 rank, for image-to-image editing (`it2i`, which additionally encodes a 512×512 source image) and text-to-image generation (`t2i`). The 768×768 `t2i` entry was measured on an earlier commit of the same branch.

Canvas	Image tokens	it2i, one per sample	it2i, packed	t2i, packed
512×512	1,024	9.05	7.25	6.50
768×768	2,304	17.45	15.76	14.55
$1{,}024 \times 1{,}024$	4,096	30.48	28.83	27.00

Table 6 shows that generation latency scales approximately linearly with the output image-token count, creating a $4\times$ cost difference between 512×512 and $1{,}024 \times 1{,}024$ canvases within the same batch. Consequently, when each data-parallel rank receives two prompt groups drawn in dataset order from a mixed-resolution prompt set, the slowest rank takes 1.16–1.84$\times$ the mean per-rank generation time, forcing all other ranks to wait at the synchronous rollout barrier (we return to this straggler bottleneck in Section 7.5).

7.3 Sibling Request Packing for Image Editing

To support group-relative credit assignment, rollout forks multiple sibling samples from each root prompt. Whereas the text-to-image BAGEL adapter already packed sibling rows into a single engine call, the original image-editing (`it2i`) adapter dispatched each sibling row as an independent request, causing the engine to recompute the vision-encoder pass and key–value conditioning context for every sibling in the group. With sibling request packing (UniRL pull request #550), whenever guidance-scale settings permit, the `it2i` adapter consolidates contiguous sibling rows from the same prompt group into a single engine request so that the engine encodes the source conditioning once and broadcasts the resulting context across the group. At 16 edits per rank (512×512), packing reduces per-rank generation time from 36.5 s to 28.4 s (22%, Table 7) at the cost of roughly 2 GiB of additional peak GPU memory, and reduces the generation time for a single eight-sibling group from 18.04 s to 13.96 s.

Because initial noise tensors are indexed per row, a packed request can span any contiguous subset of a sibling group rather than requiring the full group at once. Consequently, the adapter no longer requires `RewardStack` (Section 6.3) to round micro-batch sizes up to full group boundaries: a four-row micro-batch from an eight-sibling group executes as a single four-sample packed request.

7.4 Combined Effect of the Four Changes

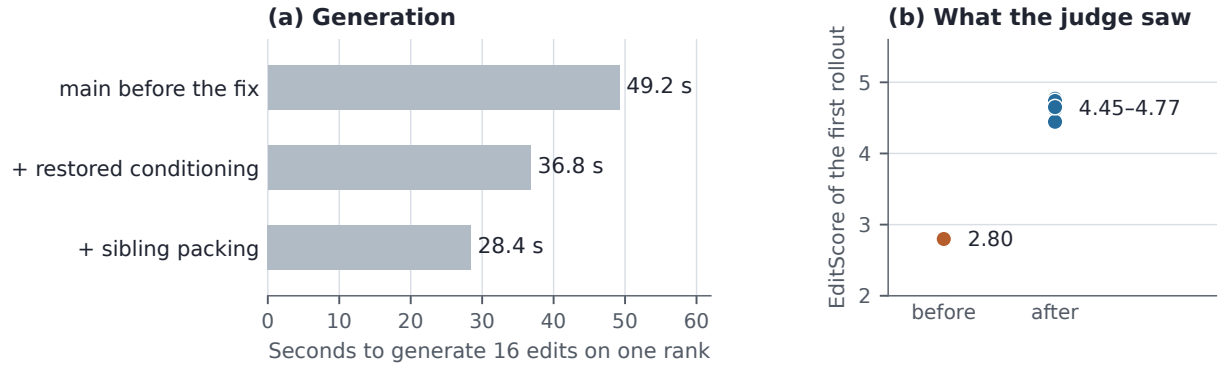


Figure 4: Cumulative effect of the four rollout-fidelity changes on BAGEL image editing (`it2i`) on one eight-GPU H20 node with a remote EditScore 8B judge. Left: per-rank generation time for 16 edits. Right: mean judge score on the first rollout prior to any optimizer step; points after the fix represent independent runs on the same initial prompt batch.

Table 7: Cumulative ablation of the four rollout-fidelity changes on BAGEL image-to-image editing (`it2i`) with LoRA on one eight-GPU H20 node (vLLM-Omni rollout, 16 prompts × 8 samples at 512×512, remote EditScore 8B judge). "Generation" reports the slowest rank's generation time for its 16 rows; "First-rollout reward" reports the mean judge score before the first optimizer update.

Rollout path	Rollouts	Generation (s)	vs. `main`	First-rollout reward
`main` before the fix	1	49.2	–	2.80
+ restored it2i conditioning (#547)	1	36.8	−25%	4.62
+ per-group canvas (#549)	1	36.5	−26%	4.77
+ sibling packing (#550)	4	28.4	−42%	4.45–4.74

Table 7 and Figure 4 summarize the cumulative impact of UniRL pull requests #547–#550 on the 512×512 image-editing workload: per-rank generation time drops by 42% (from 49.2 s to 28.4 s), while the first-rollout EditScore mean increases from 2.80 to 4.45–4.77. Across all arms, the first-update importance ratio remains within 10^{-4} of 1, and the structural token-layout check (enabled from #548 onward) passes on every replay, including configurations where `RewardStack` splits an eight-sibling group across micro-batches. As in Section 6.4, these measurements comprise one to four rollouts per arm on a single workload shape and establish the mechanism-level effect rather than multi-seed training variance.

7.5 Mitigating Mixed-Resolution Stragglers via Cost-Aware Assignment

When a rollout batch mixes output canvas resolutions, static data-parallel sharding—assigning prompts in dataset order with equal row counts per rank—creates severe load imbalance across ranks. On a mixed-resolution dataset whose canvases span a 4× range in image tokens, the default static split left at least one GPU rank idle for 29% of the text-to-image rollout window while waiting for the rank assigned the largest canvases.

To quantify how much of this straggler tail can be recovered without changing trajectory semantics, we evaluated a cost-aware work-assignment prototype. A driver-side coordinator orders prompt groups by estimated token cost and dispatches work units to ranks from a shared queue; when a rank finishes its assigned queue, work stealing allows it to claim the lowest-cost remaining unit from the busiest rank's

queue. Before credit assignment, completed rows are restored to their canonical parent-contiguous order by sample identifier I_i, leaving the trajectory structure consumed by the trainer unchanged. Table 8 reports steady-state rollout measurements across static, balanced, and work-stealing assignments.

Table 8: Comparison of rank-assignment strategies on mixed-canvas batches (second rollout of each arm on one eight-GPU H20 node, measured on the experimental load-balancing branch) for text-to-image (`t2i`) and image-to-image editing (`it2i`). "Idle" denotes the fraction of the rollout window during which at least one rank has finished generation and waits at the barrier. Except in the row-count ablation discussed in the text, the cost model estimates work by output image tokens.

Assignment	Rollout + reward (s)	Slowest rank (s)	Mean rank (s)	Idle
t2i, in-process PickScore				
Prompt-order shards (default)	150.2	147.2	103.8	29%
Cost-balanced shards	108.6	105.8	103.7	2%
Cost-balanced shards with stealing	108.7	105.8	103.6	2%
it2i, EditScore 8B on another node				
Prompt-order shards (default)	99.3	91.0	63.7	29%
Prompt-order shards, reward overlapped	93.7	91.2	63.7	30%
Cost-balanced shards with stealing	77.4	68.0	63.7	6%
Cost-balanced with stealing, reward overlapped	71.5	68.1	63.8	5%
it2i, EditScore 72B on another node				
Prompt-order shards (default)	128.4	90.8	63.7	21%
Cost-balanced with stealing, reward overlapped	131.1	68.4	63.8	8%

Table 8 and the companion two-rollout traces yield three findings:

1. **Static cost balancing eliminates most straggler idle time when the cost model is accurate.** In text-to-image (`t2i`) generation, cost-balanced assignment reduces combined rollout-plus-reward time from 150.2 s to 108.6 s and cuts the barrier idle fraction from 29% to 2%, while mean per-rank generation time remains unchanged at approximately 104 s, confirming that work is redistributed across ranks rather than reduced.
2. **Work stealing provides robustness against cost-model misspecification.** Under the image-token cost model, dynamic stealing was never triggered in `t2i` and transferred at most 2 prompt groups per rollout in `it2i`. Under a deliberately misspecified cost model that counts only rows and ignores canvas resolution—forfeiting 25 s of the `t2i` balancing gain—work stealing recovers 18 s of that penalty, reducing combined rollout-plus-reward time from 133.9 s to 115.9 s.
3. **Cost-aware assignment composes with generation–reward overlap only when the judge has spare capacity.** In `it2i` with the 8B EditScore judge, generation–reward overlap alone reduces combined rollout-plus-reward time from 99.3 s to 93.7 s, cost-balanced assignment with stealing reduces it to 77.4 s, and combining both mechanisms reduces it to 71.5 s (28% total reduction).

By contrast, when paired with the saturated single-replica 72B EditScore judge, cost-aware assignment *increases* combined rollout-plus-reward time relative to the static baseline: in the steady-state rollout, duration rises from 128.4 s to 131.1 s with overlap (and to 132.2 s without overlap); in the first rollout, it rises from 118.3 s to 131.5 s with overlap (and to 133.3 s without overlap). Although cost balancing still reduces the slowest rank's generation time from 90.8 s to 68.4 s, equalizing rank completion times causes all eight ranks to submit their final reward requests nearly simultaneously, concentrating the judge queueing delay that the imbalanced static baseline had naturally staggered. This is the converse of the request-staggering effect analyzed in Section 6.4 and implies that generation load balancing should be gated on reward-service headroom (or paired with additional judge replicas and cross-request batching).

A second interaction remains unaddressed in this prototype: in every arm that reordered prompt groups across ranks, the subsequent training phase ran 1–5% slower in the steady-state rollout and 3–12% slower in

the first rollout. Control runs that routed dataset-order shards through the same coordinator queue without cross-rank reassignment incurred a 2–3% training slowdown, indicating that part of the overhead stems from the queue path itself and the remainder from cross-worker `TensorRef` transfers when completed rows are restored to canonical parent-contiguous order before training. While total iteration time in the steady-state rollout remained shorter whenever balancing reduced rollout-plus-reward time, in the first rollout under the 8B judge without overlap the training slowdown outweighed the rollout gain, increasing total iteration time by 2.7 s. Propagating the cost-balanced shard assignment directly into the training phase to avoid cross-worker reordering remains future work.

8 Evaluation: Reported Results and Remaining Evidence

We organize the main evaluation around four research questions (Table 9). Throughout the synchronous training and systems benchmarks below, one outer-loop iteration collects one rollout batch and applies the configured optimizer updates; we therefore refer to training progress in *rollouts* (and to per-rollout wall-clock duration in systems benchmarks as *iteration* or *step* time). The prespecified artifact contract requires each run to archive its resolved configuration, source commit and patch state, model and dataset revisions, environment manifest, structured metrics, and evaluation outputs (Appendix C); the paragraph below notes where current bundles remain incomplete. The prespecified protocol requires primary comparisons to run one framework at a time on the same reserved hardware; E3 follows this single-node protocol, whereas the two E3-AR runs executed concurrently on separate two-node H20 allocations in the same cluster (Section 8.2).[1]

Evidence scope and date. The evaluation draws on dated artifact bundles across two evidence tiers. Results for E3, E5-T, and E6 come from the legacy online experiment report (retrieved on 2026-09-27 with an evidence cutoff of 2026-09-22) combined with the companion report pull request #2 at commit `ecce5097`; source snapshots, transcribed summaries, and a deterministic figure and table generator are committed under `artifacts/reported_results/2026-09-22/`. Those numbers are *reported summaries*, not newly inspected training logs. Because the underlying per-run directories were unavailable during that integration, their source identities, complete timing distributions, and admission gates have not been independently revalidated here, and we do not reconstruct learning curves or uncertainty bars for them. The remaining bundles contain artifacts exported directly from the experimental runs. The later E3-AR study was rerun and is archived separately under `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/`, including resolved recipes, the recipe and effective-work audit, run identities, post-warm-up per-rollout records, phase and band summaries, and the deterministic plotting script. The HunyuanImage-3.0 and MiniMax-H3 recipe reproductions in Section 8.1 are likewise generated directly from their exported W&B histories and resolved training recipes. Results for E0, E1, E2, and E4 come from `artifacts/reported_results/2026-10-08-e0-e1-e2-e4/`, which archives data exported from those runs: per-prompt endpoint scores, per-rollout training metrics, internal validation series, pre-launch parity checks, and a training/evaluation overlap audit for E1; per-checkpoint judge aggregates for E2; the preregistered analysis output for the corrected E4b rerun (denoted E4 v2) alongside the earlier E4b analysis; and raw parity records for E0. The bundle's generator script renders all tables, figures, and numerical macros quoted for those four studies (except E4a's synthetic check count, which comes from the 2026-09-22 bundle), and its README documents the evidence tier of each component. Generated images, text completions, raw logs, and model checkpoints remain on cluster storage, so the full run-directory contract of Appendix C is not yet met. We retain the distinction between the two tiers and list unresolved source discrepancies for the reported summaries in `EXPERIMENT_RESULTS.md`. Unless explicitly identified as an observed measurement, protocol descriptions below state the prespecified experimental design; matching each setting to a resolved run configuration remains an artifact-release obligation.

[1] Throughout the paper, we follow each baseline project's official capitalization: *veRL* for the language-model post-training framework (Sheng et al., 2024) and *VeRL-Omni* for the multimodal and diffusion extension (Huang et al., 2026).

Table 9: Experiment-to-claim map and evidence status through 2026-10-14. E3, E5, and E6 are as reported on 2026-09-22; E3-AR follows the 2026-10-11 bundle, and E0, E1, E2, and E4 follow the 2026-10-08 bundle. P0 rows remain necessary for the primary trajectory-centric claim.

ID	Priority	Controlled study	Paper evidence
E0	P0 gate	Correctness, configuration, and instrumentation checks	Parity measured on five publication paths and on every E1 rollout; no tolerance set; evidence packaging remains incomplete
E1	P0	Qwen3-4B-Base on DAPO-Math	GRPO below base at the registered endpoint in all three seeds; DRPO (added later) above base in all three as graded, with one seed mostly reaching the token limit
E2 / E2-R	P0	SD3.5-Medium diffusion training (UniRL E2 and VeRL-Omni reference E2-R)	Three-seed UniRL scores and diversity reported (E2); VeRL-Omni reference trained, not evaluated (E2-R)
E3	P0	Native UniRL–VeRL-Omni recipes, eight GPUs	Aggregate timings reported; strict semantic parity not established
E3-AR	P0	Recipe-matched Qwen3-4B-Base GRPO, UniRL–veRL, 16 GPUs per system	500 rollouts complete; resolved recipes, parity audit, per-rollout measurements, and curves released
E4 (E4a/E4b)	P0	Root versus rewrite grouping: oracle check (E4a) and 3-seed training (E4b)	Synthetic and recorded-rollout equality checks pass; corrected rerun inconclusive at three seed pairs
E5 (E5-R/E5-T)	P0 minimum	Paired IR representation cost (E5-R) and colocated vs. separate deployment topology (E5-T)	Deployment topology measured (E5-T); paired representation cost not measured (E5-R)
E6 (S0/D0/A1)	P1	Qwen3-4B-Base colocated sync (S0), separate sync (D0), and async overlap (A1)	Single-seed pilots only; formal sweep canceled

The execution priority is E0, followed by E1–E4 and the minimum E5 topology/cost slice (E5-T and E5-R); E6 and broader E5 scaling/transport backends follow. When compute is constrained, completing primary E0–E5 takes precedence over adding models, algorithms, or scaling points: without E4 the evaluation would not directly test cross-stage lineage, and without E5-R and E5-T it would not bound the cost of that abstraction.

Environment and source identity. E3 and the deployment-topology comparison E5-T use a fixed allocation of eight GPUs; complete hardware, interconnect, environment, model, data, and scorer manifests remain to be attached to those legacy summaries. E3-AR uses two eight-GPU H20 nodes per framework, executed concurrently on disjoint node pairs in the same cluster rather than sequentially on the same hosts, and releases its pinned source commits and resolved recipes alongside the per-rollout records. The 2026-10-08 bundle records the source commits and hardware allocations for E0, E1, E2, and E4: E1 used four nodes of eight NVIDIA H20 GPUs (32 GPUs total), and E2 and E4 each used one eight-GPU H20 node per run. Hardware specifications from these H20 allocations are not extrapolated to the unverified eight-GPU E3 summary, and the v0.2.0 design-audit commit (`fd344ad`) is kept distinct from the commits used by experimental runs.

8.1 RQ1: Does UniRL learn on representative workloads?

The first question is scientific rather than purely operational. A complete training-validity claim requires improvement over the frozen base together with satisfactory collapse diagnostics. E2 reports an improvement in automatic scores but triggers the diversity guard, so its result is a qualified quality–diversity trade-off. E1's prespecified GRPO recipe fails the first condition at its registered endpoint; a Divergence-Regularized Policy Optimization (DRPO) (Yao et al., 2026) recipe added later meets the first condition in all three seeds, but one seed fails the second because most of its final-checkpoint answers run to the generation token limit. Reproducing another implementation additionally requires a full-budget, mathematically aligned reference with common quality endpoints; a systems speed run alone cannot supply that evidence. The VeRL-Omni reference training run (E2-R) has been trained across three seeds but has not been evaluated head-to-head on held-out prompts, while the adapted AR systems launcher (E3-AR) has not yet been evaluated on the aligned external quality benchmark.

AR workload. E1 trains Qwen3-4B-Base (Yang et al., 2025) with full-parameter GRPO on DAPO-Math-17k (Yu et al., 2025). The dataset name does not imply use of the DAPO optimizer; we fix the population-standard-deviation group estimator implemented in the audited recipe. Each rollout collects 64 prompts with

eight samples per prompt (512 trajectories) via SGLang (TP=1) with thinking mode enabled, temperature 1.0, top-p 1.0, top-k disabled, and a maximum of 8,192 new tokens. The learner uses group-standard-deviation normalization, symmetric clipping at 0.2, per-sequence token-mean loss (`seq-mean-token-mean`), four optimizer updates per rollout under a 10,240-token per-GPU micro-batch budget, AdamW (learning rate 10^{-6}, weight decay 0.01), and full-weight publication every rollout. We prespecify 800 rollouts on 32 GPUs, internal AIME validation every ten rollouts, full checkpoints every 200 rollouts, and seed IDs 11, 22, and 33 intended to govern both data order and the SGLang server random number generator (RNG). These seed IDs and the checkpoint cadence are passed as run-time overrides because the stock recipe defaults to data-source seed 42; at the audited release, per-request SGLang sampling seeds require deterministic inference mode, so the protocol specified seeding the server RNG via engine arguments (see the deviation noted below). The prespecified external evaluation scores the frozen base model and checkpoints at rollouts 200, 400, 600, and 800 on MATH-500 (500 problems, avg@4) and AIME 2024/2025 (30 problems each, avg@16) with the `math_verify` grader under fixed decoding parameters (including a 16,384-token generation limit, twice the training cap, at a lower temperature). The UniRL run and the reference baseline must align prompts, chat template, verifier, maximum generation length, group size, effective batch, clipping, KL treatment, optimizer updates, and evaluation decoding. The public veRL Qwen3 FSDP example serves only as a starting scaffold because its stock model variant, dataset, group size, response budget, and KL/objective settings differ. E3-AR uses a pinned adapted launcher whose resolved recipes are archived and checked field by field against fifteen cross-framework effective-work identities in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/recipe_parity_audit.json`; two residual estimator-level differences that configuration cannot remove are quantified there, and the bundle documentation lists the recipe items outside those fifteen identities. Absent frozen-base and checkpoint evaluations under the common external protocol, E3-AR does not by itself establish quality reproduction. The final 800-rollout checkpoint is primary. Internal evaluation uses AIME 2024/2025, so an AIME-selected best checkpoint is reported only as validation-selected and cannot establish an untouched AIME test result. MATH-500 is the primary external AR endpoint, with a training/evaluation overlap audit required before interpretation; the fixed-budget AIME curves are secondary diagnostics. The protocol calls for every seed, final-checkpoint curves, and 95% confidence intervals; avg@k denotes mean accuracy across k sampled answers, not pass@k.

Table 10: E1 external evaluation of the frozen base and every surviving checkpoint. MATH-500 avg@4 at the final checkpoint is the registered endpoint; ± is the sampling standard error over the fixed prompt set, not seed uncertainty. †Mean of two independent evaluations of the same checkpoint. Checkpoints 200 and 400 of GRPO seed 11 were lost before evaluation; the seed 11 rerun, a separate run under the same seed ID whose last checkpoint is at rollout 400, has its own row. DRPO was added after the first GRPO seed.

	MATH-500 avg@4 at rollout				AIME avg@16 at 800	
Run	200	400	600	800	2024	2025
Frozen base (no training)			0.6695 ± 0.0076		0.0729	0.0500
GRPO, seed 11	–	–	0.6205 ± 0.0058	0.5920 ± 0.0040†	0.0344†	0.0062†
GRPO, seed 11, rerun	0.8100 ± 0.0054	0.7875 ± 0.0050	–	–	–	–
GRPO, seed 22	0.8145 ± 0.0052	0.6515 ± 0.0062	0.4455 ± 0.0064	0.3030 ± 0.0064	0.0000	0.0042
GRPO, seed 33	0.7915 ± 0.0052	0.7515 ± 0.0052	0.6790 ± 0.0052	0.6315 ± 0.0056	0.0271	0.0229
DRPO, seed 11	0.8435 ± 0.0044	0.8675 ± 0.0045	0.8605 ± 0.0046	0.8488 ± 0.0031†	0.1698†	0.1792†
DRPO, seed 22	0.8710 ± 0.0043	0.8570 ± 0.0048	0.8440 ± 0.0046	0.8120 ± 0.0050	0.1792	0.1167
DRPO, seed 33	0.8645 ± 0.0048	0.8310 ± 0.0050	0.8585 ± 0.0046	0.8240 ± 0.0051	0.1667	0.1479

E1 result: GRPO gains early and ends below the base model. All three prespecified GRPO seeds completed 800 rollouts, and every surviving checkpoint was scored on the external benchmarks (Table 10, Figure 5). At rollout 200 the two seeds with a saved checkpoint score 0.8145 and 0.7915 on MATH-500, against 0.6695 for the frozen base. Every later checkpoint is lower than the one before it in every seed, and the final checkpoints score 0.5920, 0.3030, and 0.6315. No seed improves on the base at the registered endpoint, so E1 does not show the improvement over the frozen base that a training-validity claim requires. The three final deficits relative to the base model range from 0.0380 to 0.3665 (mean 0.1607), with a seed-level 95% confidence interval of [−0.61, +0.28] for the net change from base; all three seeds agree on the negative direction, though with two degrees of freedom the interval is too wide to bound the magnitude tightly. Mean training reward declines over the same span in every seed—from 0.375, 0.420, and 0.405 over rollouts 101–200

to 0.183, 0.079, and 0.216 over rollouts 701–800—ruling out classic overfitting where training reward rises while held-out accuracy falls. Responses from the final checkpoints also fail to terminate more frequently: 10.6%, 30.3%, and 20.0% of their MATH-500 completions reach the 16,384-token generation limit, compared with 3.0% for the frozen base, and at most 6.4% of those truncated responses are graded correct. (We classify a response as reaching the limit when its stored text, once re-tokenized, spans at least 97% of the 16,384-token cap.) Non-termination accounts for part, but not all, of the accuracy decline. Between rollouts 200 and 800, seed 22's share of responses at the limit rises from 9.4% to 30.3% while accuracy on the remaining responses falls from 0.893 to 0.407; seed 33's share at the limit rises from 4.6% to 20.0% while accuracy on the remaining responses falls from 0.825 to 0.773. Holding accuracy on terminating responses fixed at its rollout-200 level, increased non-termination accounts for 34% of the overall drop in seed 22 and 74% in seed 33; because non-terminating responses are not a random subset of prompts, this accounting decomposes the observed score drop rather than isolating a causal mechanism.

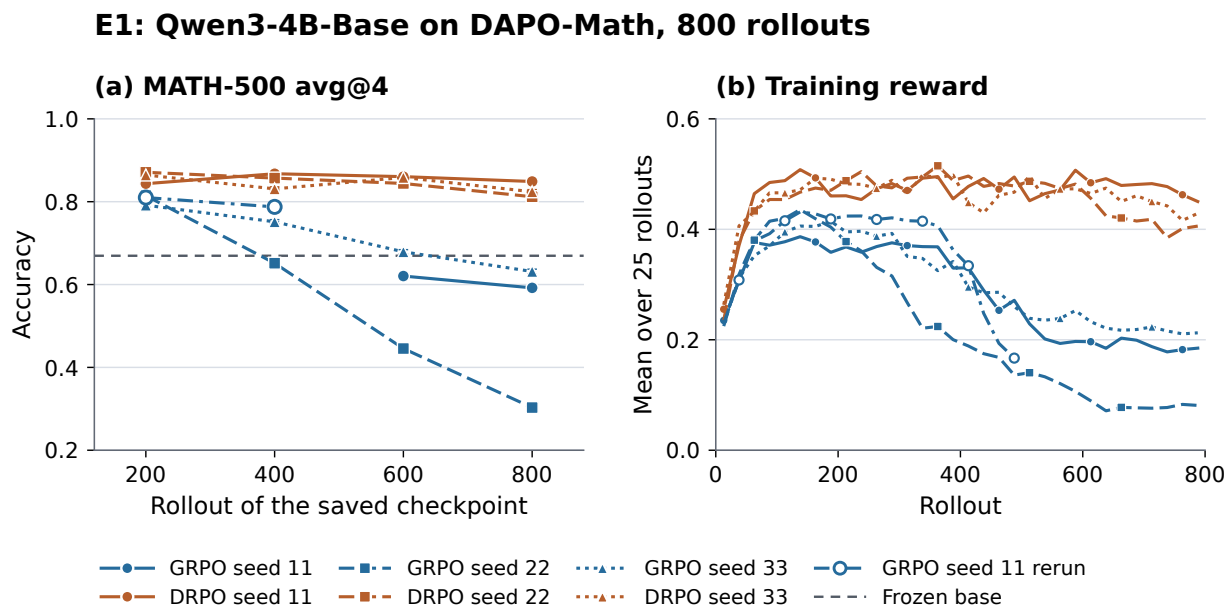


Figure 5: E1 on the registered external endpoint and during training. (a) MATH-500 avg@4 at each saved checkpoint (one curve per training run, dashed horizontal line for the frozen base; error bars denote prompt-sampling standard errors). (b) Mean training reward over consecutive 25-rollout blocks for the same runs. Color indicates the algorithm arm, while marker shape and line style indicate the seed; the seed 11 rerun, which trained to rollout 518 with its last saved checkpoint at rollout 400, is plotted with hollow markers and a dash-dot line and is not connected to the original seed 11. Each curve represents an individual run; no cross-seed mean is plotted because only two formal GRPO seeds have surviving checkpoints at rollouts 200 and 400.

A DRPO arm added after the first GRPO seed. After GRPO seed 11's internal validation accuracy fell from a peak of 0.126 to 0.018 at rollout 800—below its initial 0.021—we added a DRPO recipe (Yao et al., 2026) (configured as `spo_adaptive_eps`) under the same compute budget, dataset, hardware allocation, and seed IDs. Adding this arm departs from the prespecified protocol, which registered only GRPO; the comparison plan between the two arms was recorded when DRPO seeds 22 and 33 were launched, before their outcomes were observed. All three DRPO seeds completed 800 rollouts, and every saved DRPO checkpoint scores 14.2 to 20.2 percentage points above the frozen base on MATH-500. Their final checkpoints score 0.8488, 0.8120, and 0.8240, yielding a seed-level 95% confidence interval of [+0.11, +0.21] for the gain over base. Under the default grader, every DRPO seed finishes above every GRPO seed—the strongest rank ordering possible with three seeds per arm ($p = 0.05$, exact one-sided Mann–Whitney test over all $\binom{6}{3} = 20$

assignments). However, that final-checkpoint ordering hinges on how non-terminating responses are scored in one run. In DRPO seed 22, 74.9% of final MATH-500 answers reach the 16,384-token limit (versus at most 0.7% in the other two DRPO seeds), yet 75.3% of those truncated responses are graded correct because `math_verify` accepts a boxed answer anywhere in the completion. Under a stricter post-hoc scoring rule that counts every limit-reaching answer as incorrect, the final DRPO scores become 0.8485, 0.2480, and 0.8235 versus 0.5878, 0.2835, and 0.6190 for GRPO, placing DRPO seed 22 below the base model and yielding $p = 0.35$. At rollouts 200, 400, and 600, every DRPO seed exceeds every GRPO run (including the seed 11 rerun) under either scoring rule (Table 21). At rollouts 400 and 600, only 9.7% and 1.6% of DRPO seed 22's MATH-500 answers hit the 16,384-token evaluation limit, even though 87.7% of its training rollouts from rollout 301 onward reached the 8,192-token training cap (Appendix E), because external evaluation permits twice the generation length at a lower decoding temperature. Finally, the DRPO configuration differs from the GRPO configuration across five settings—the policy-gradient loss function, omission of group-standard-deviation advantage normalization, token-level loss aggregation, use of rollout-engine log probabilities as the π_{old} anchor, and micro-batch shard balancing—so this comparison evaluates two complete recipes rather than isolating the objective alone.

E1 deviations and limits. In practice, seed IDs controlled only the training data order; rollout generation sampling was left unseeded, departing from the prespecified server-RNG seeding. Evaluation decoding on the text benchmarks was also unseeded; consequently, the final checkpoints of GRPO seed 11 and DRPO seed 11 were each evaluated twice, and Table 10 reports their mean. During AIME evaluation, the requested max-new-tokens budget was clipped to fit within the model's context window. GRPO seed 11 and DRPO seed 22 were each resumed from their rollout-200 checkpoints, restoring model weights, optimizer state, and data-loader position (DRPO seed 22's initial allocation ended at rollout 369, and rollouts 201–369 from that first segment were discarded). Because GRPO seed 11's checkpoints at rollouts 200 and 400 were lost from storage prior to evaluation, we launched a rerun under the same commit, dataset, and data order, scoring its checkpoints at rollouts 200 and 400 (0.8100 and 0.7875 on MATH-500) before resuming it from rollout 400 toward 800; that allocation ended at rollout 518, prior to checkpoint 600, leaving no later endpoint score. Because generation sampling is unseeded, this rerun is an independent trajectory rather than an exact replay: its internal validation accuracy lies above the original seed 11 at every evaluation from rollout 70 to 420 and below it from rollout 430 onward, finishing at 0.004 at rollout 510—the lowest of the four GRPO runs at that step (Appendix E). We therefore report the rerun in a separate row in Table 10 and do not connect it to seed 11's later checkpoints. Because no external reference framework was run to rollout 800 and evaluated on MATH-500 (E3-AR in Section 8.2 runs for 500 rollouts without external benchmark scoring), we cannot isolate whether the late-training GRPO degradation stems from the recipe or from the system implementation. The required training/evaluation overlap audit identified 9 of the 500 MATH-500 problems among the 17,917 training prompts (8 verbatim after normalization and 1 near-verbatim) and zero overlapping AIME problems; excluding those 9 problems alters no checkpoint's change from base by more than 0.0057. An additional 20 MATH-500 problems share at least half of their normalized 30-character substrings with a training prompt and were retained. Finally, Appendix E details the internal AIME validation series (evaluated every ten rollouts under training decoding settings) and the checkpoints it selects, along with token-limit truncation breakdowns, training diagnostics, and aborted launches excluded from Table 10.

Diffusion workload. E2 fixes SD3.5-Medium (Stability AI, 2024) and the UniRL Flow-GRPO setting used by E3, i.e. the stock `sd3_vllmomni.yaml` recipe with the alignment overrides in `run_unirl_sd35_aligned.sh`: 48 prompts with 16 images (768 samples) per rollout, 384×384 training resolution, ten denoising steps, three SDE steps in the first half, $\eta = 0.8$, guidance 1, and distinct initial noise. PickScore (Kirstain et al., 2023) supplies the training reward; advantages subtract each prompt-group mean and divide by the batch-wide reward standard deviation. LoRA rank/alpha is 32/64 on eight matched attention projections; the learner uses two updates, micro-batch size eight, learning rate and weight decay 10^{-4}, clip 10^{-5}, and weight publication every rollout. We prespecify 300 rollouts on one eight-GPU node with seed IDs 11, 22, and 33. The same budget and seed IDs apply to the pinned VeRL-Omni reference run (E2-R); checkpoint export and estimator parity are audited before admission. Because the stock configurations differ in SDE transition strategy—UniRL uses `FlowSDEStrategy` whereas VeRL-Omni uses Coefficients-Preserving Sampling (CPS)—as well as step selection and normalization/anchor semantics, E2

and E2-R currently define a related-recipe comparison. At least one full-budget E1 or E2 reference must pass mathematical and work parity before the paper claims reference-implementation reproduction. Frozen base and every 50-rollout checkpoint use a shared diffusers evaluation pipeline. The primary image endpoint is PartiPrompts HPSv3 (Ma et al., 2025b); ImageReward (Xu et al., 2023) and compositional scores provide complementary checks. These three metrics use held-fixed automatic judges distinct from the optimized PickScore model rather than human evaluations; Table 11 and Figure 6 also report held-out PickScore on PartiPrompts to track progress on the training objective itself.

Image evaluation identity. The registry key `image/geneval2` points to UniRL's `synthetic/test.jsonl` (800 prompts) and a Qwen3-VL-8B Soft-TIFA-style compositional VQA scorer, served by the reward service under the `vqascore` endpoint key by default or run in-process with `-local-geneval2`. We refer to this benchmark as the *synthetic compositional set* (labeled `Composition (VQAScore)` in Table 11 and Figure 6); it is not an official GenEval 2 benchmark (Kamath et al., 2025) without a separate data-and-scorer parity audit. All image evaluations generate 512×512 images over 40 denoising steps with guidance scale 1.0 and base evaluation seed 42 shared across training seeds and checkpoints. The PartiPrompts preference metrics (HPSv3, ImageReward, and PickScore) were evaluated on 16 images per prompt across all 1,632 prompts—matching the sample count required by the LPIPS diversity guard—rather than the prespecified single image per prompt. Because image seeds are indexed deterministically by prompt and sample index independently of the total sample count, the prespecified single-image endpoint corresponds to the first-sample subset of these generations, which has not yet been scored separately. The synthetic compositional endpoint uses one image per prompt (800 images), an explicit linear sigma grid, prompt-hashed seeds, and a 256-token text-encoder limit, whereas the PartiPrompts preference endpoint uses the pinned pipeline schedule and index-based seeds. Dataset sizes, revisions, overlap counts, and scorer hashes remain open manifest fields.

The diversity guard is within-prompt mean pairwise LPIPS (Zhang et al., 2018) (AlexNet features), computed over 16 images per PartiPrompts prompt at the same evaluation setting, with identical image seeds for base and checkpoint. We average all 120 pair distances per prompt, then report the relative change in the mean across prompts and a paired prompt-bootstrap interval. A greater than 10% relative decrease is a study-specific investigation trigger; it is not a literature-established quality threshold. A near-zero base makes this relative criterion undefined and requires the absolute change to be reported. This guard diagnoses reduced diversity and is not itself a perceptual-quality metric.

Table 11: E2 diffusion endpoints after 300 rollouts, with three training seeds. Base and rollout-300 columns are levels, the latter averaged over seeds; changes are absolute score differences unless marked as relative percentages. Intervals are 95% t intervals over the three seeds, with two degrees of freedom. The LPIPS row reports the relative change in within-prompt diversity; its rollout-300 level is not in the bundle and is not reconstructed.

Metric	Base	Rollout 300	Δ score	95% interval	Δ relative
HPSv3	1.2024	9.8126	+8.6102	[+8.3831, +8.8373]	–
ImageReward	−0.2784	1.1148	+1.3932	[+1.3147, +1.4718]	–
PickScore	0.8021	0.8991	+0.0970	[+0.0936, +0.1005]	–
Composition (VQAScore)	0.1011	0.1552	+0.0541	[+0.0473, +0.0608]	+53.5%
LPIPS diversity	0.7987	–	–	[−26.58%, −22.98%]	−24.78%

E2 result: higher automatic scores with lower diversity. The mean HPSv3 gain over three seeds is 8.6102, with a seed-level 95% interval of [8.3831, 8.8373], while within-prompt LPIPS changes by −24.78%, with an interval of [−26.58%, −22.98%]. Figure 6 shows every seed at the frozen base and the six saved checkpoints, keeping score improvement and diversity reduction adjacent. All evaluations completed with zero scoring errors across the 1,632 PartiPrompts prompts (16 images per prompt) and 800 synthetic compositional prompts (one image per prompt). All three training seeds exceed the prespecified 10% LPIPS diversity-drop threshold. By rollout 50, within-prompt LPIPS has already fallen by −14.23% (57% of its rollout-300 reduction) while HPSv3 has reached 86% of its final gain; stopping at rollout 50 therefore does not avoid the diversity loss, although earlier stopping points or explicit diversity regularizers remain untested.

Because the held-out judges are automated neural scorers, their gains do not establish human preference or rule out shared reward exploitation. The supported conclusion is therefore a quality–diversity trade-off rather than unqualified training validity. In Figure 6, LPIPS diversity was evaluated only at rollouts 50 and 300, and the compositional score trajectories fluctuate more across intermediate checkpoints than the PartiPrompts curves because compositional scoring averages over 800 single-image prompts rather than $1{,}632 \times 16$ images.

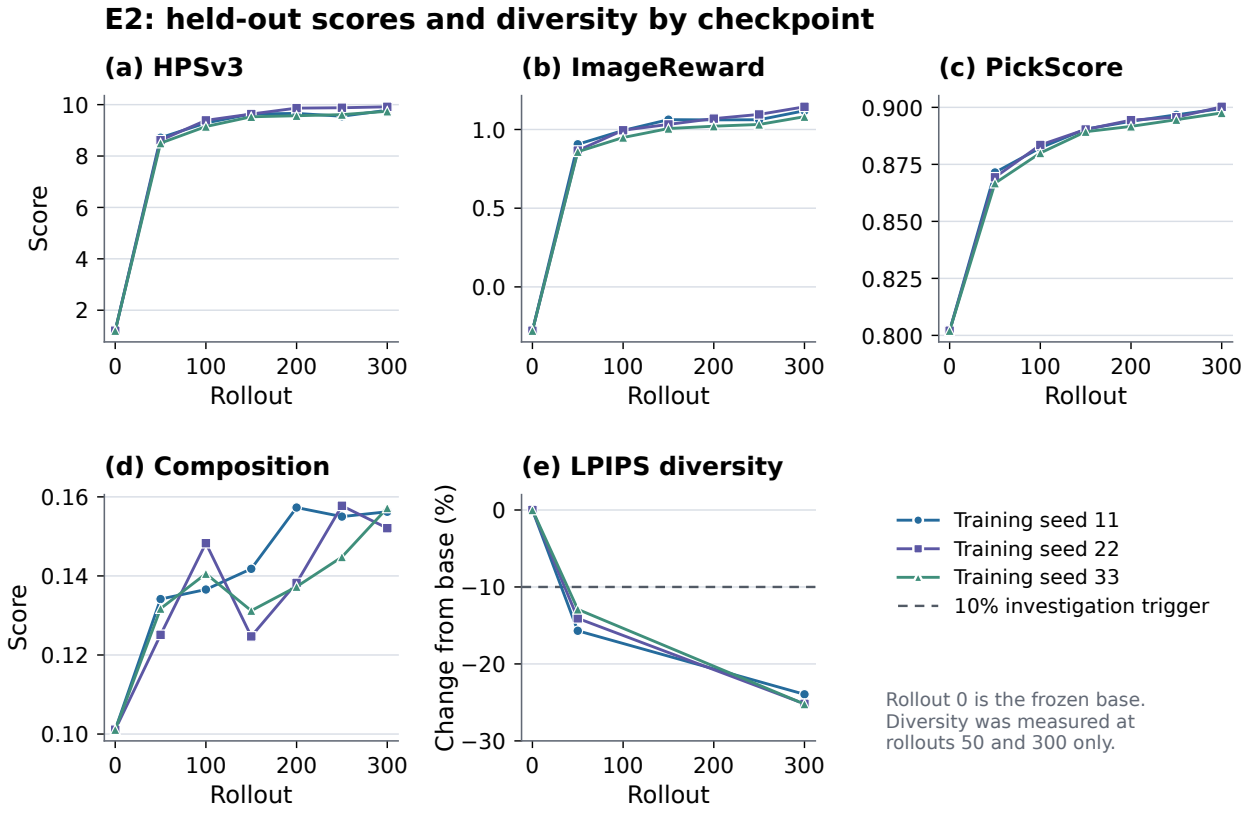


Figure 6: Per-seed E2 trajectories from the frozen base (rollout 0) through the six saved checkpoints. Panels (a)–(d) show judge scores in their own units; PickScore is the optimized reward and the other judges are held fixed. Panel (e) shows the relative change in within-prompt LPIPS, measured at rollouts 50 and 300 only, with the 10% investigation trigger as the dashed line. Lines are training seeds, not individual images.

Reference comparison (E2-R) remains open. The VeRL-Omni reference run E2-R completed three training seeds and verified checkpoint exports, but no held-out head-to-head evaluation was performed. Export correctness cannot establish quality parity, and evaluation alone would not remove the native flow versus CPS estimator mismatch. An explicitly labeled native-recipe comparison could provide context; strict reproduction still requires mathematical and effective-work parity.

HunyuanImage-3.0 recipe reproduction. We further evaluate UniRL on HunyuanImage-3.0 (using the `HunyuanImage-3.0-Instruct` checkpoint) as a high-resolution text-to-image workload. The run is single-stage diffusion: the train-side recipe samples and replays only the image-generation mode, with no AR rollout or AR objective, so it does not exercise the shared-backbone AR and diffusion trajectory described in Sections 3.4 and 3.5. It uses one node with eight H20 GPUs, 16 prompts and four samples per prompt, producing 64 images per rollout at $1{,}024 \times 1{,}024$ resolution. Sampling uses 16 inference steps and three SDE steps, and training uses Flow-GRPO with PickScore reward and seed 42. The exact resolved recipe is archived under `artifacts/reported_results/2026-09-30-hunyuanimage3/`.

We report the run through its durable checkpoint at rollout 360; the archived history continues to rollout 369. Mean training reward increases from 0.762 over the first ten rollouts to 0.870 over rollouts 351–360 (Figure 7). This result shows that the HunyuanImage-3.0 text-to-image recipe executes and improves its training reward in UniRL. Because the run uses one seed and has no held-out image-quality evaluation, we treat it as recipe-reproduction evidence rather than a comparative quality result.

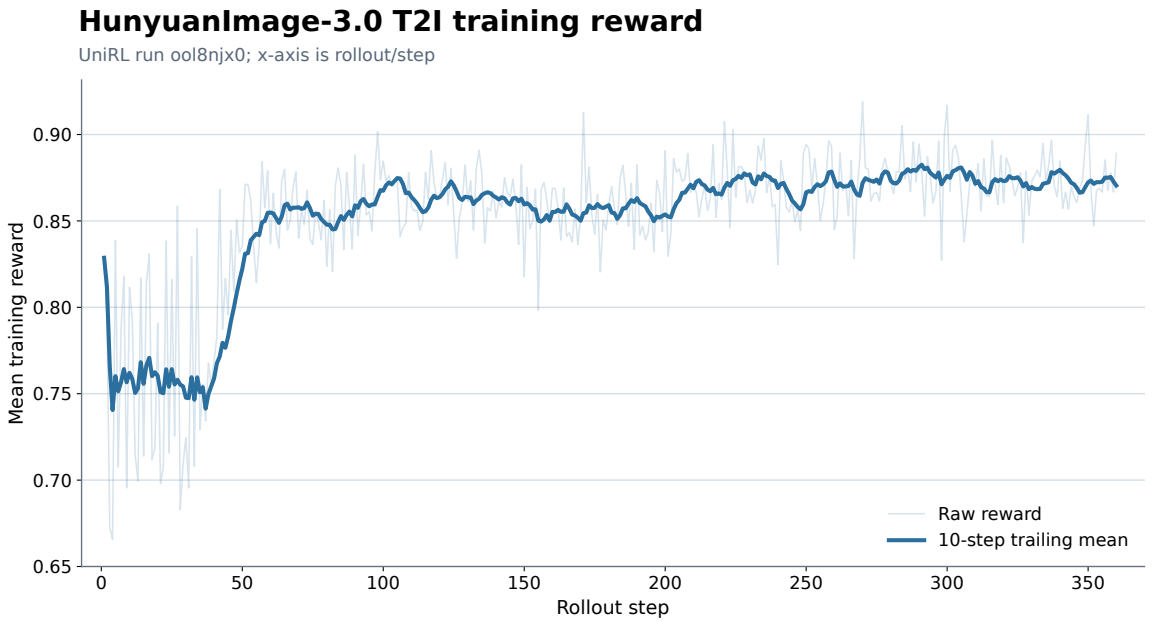


Figure 7: HunyuanImage-3.0 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL's rollout counter (`rollout/step`), rather than W&B's default `_step`.

MiniMax-H3 recipe reproduction. We also evaluate UniRL on MiniMax-H3 text-to-video-with-audio generation. Each rollout contains eight prompts with 16 samples per prompt, or 128 trajectories, generated at 256×384 resolution with 124 frames and ten denoising transitions. Training uses one optimizer update per rollout, learning rate 10^{-4}, ImageBind and CLAP rewards, seed 42, and one node with eight H20 GPUs. The ImageBind term scores audio–video correspondence on the middle frame and the CLAP term scores text–audio agreement; neither scores prompt–video alignment or temporal video quality. The resolved configuration is archived under `artifacts/reported_results/2026-09-30-minimax-h3/`.

We report the run through its durable checkpoint at rollout 200; the archived history continues to rollout 204. Mean training reward increases from 0.194 over the first ten rollouts to 0.375 over rollouts 191–200 (Figure 8). Because this is a single-seed run with no held-out video and audio quality evaluation, we use it as evidence of executable recipe and model coverage rather than as a comparative-quality, systems-performance, or time-to-quality result.

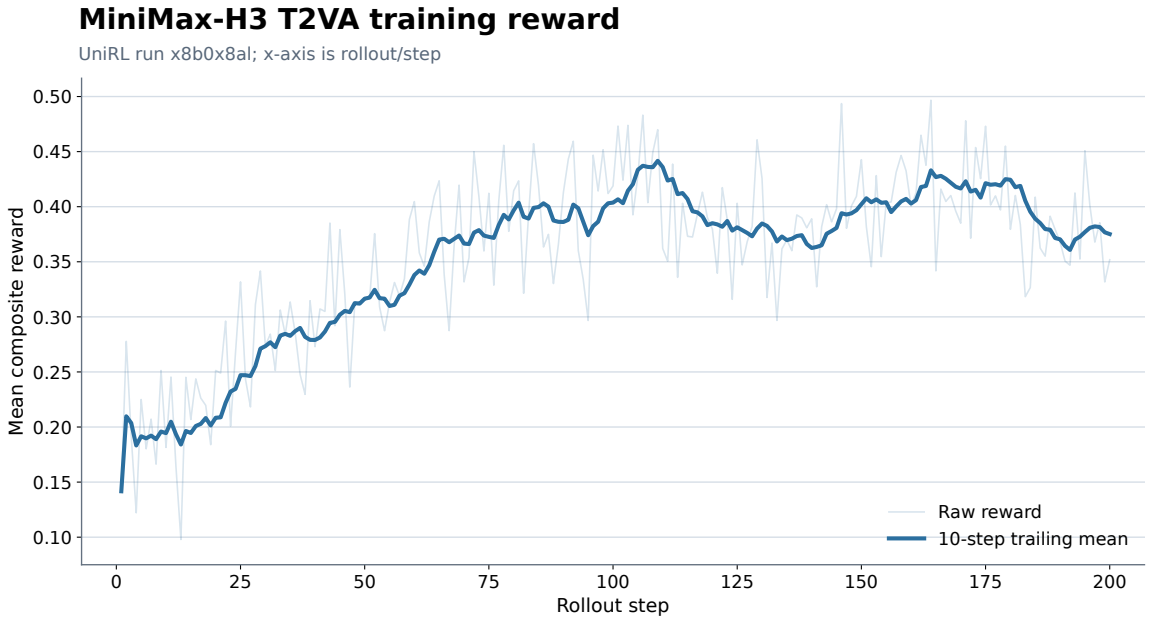


Figure 8: MiniMax-H3 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL's rollout counter (`rollout/step`), rather than W&B's default `_step`.

8.2 RQ2: What is end-to-end system performance?

We report two complementary performance comparisons. E3 compares UniRL with VeRL-Omni commit `01c87ee5` for the E2 SD3.5-Medium shape-matched work unit on one reserved eight-GPU node (Huang et al., 2026); E3-AR is a separately rerun, recipe-matched Qwen3-4B-Base GRPO comparison against veRL. The planned E3 work unit matches E2: 48 prompts with 16 samples per prompt (768 images per rollout), 384×384 resolution, ten denoising steps with three early-window SDE steps, two optimizer updates per rollout, micro-batch size eight, LoRA rank/alpha 32/64, learning rate and weight decay 10^{-4}, clipping at 10^{-5}, and PickScore reward. Because the two frameworks target different attention projection sets in their stock LoRA configurations, matching rank and alpha does not yield identical trainable parameter counts. The reported protocol specifies three process-level repetitions of 30 rollouts per configuration, discarding the first five rollouts as warm-up and retaining at least 20 timed rollouts per repetition. Each system runs in isolation on the same reserved node, though cache-warming policy and randomized run order remain unverified pending access to the raw run logs. Table 12 reports the three distinct configurations: UniRL with its native flow strategy and scaled dot-product attention (SDPA), and VeRL-Omni with its native CPS strategy under both SDPA and FlashAttention-3 (FA3). The stock UniRL recipe uses `FlowSDEStrategy` and three sampled early indices; the VeRL-Omni launcher requests `cps` and an early three-step window. These differences change the transition distribution and likelihood, not merely the kernel implementation. Normalization, anchors, actual replay work, and reward-service GPU placement also require an audit. At the audited release the stock SD3 recipes also enable update-membership shuffling (`shuffle_updates`), which the reported runs predate; a rerun at v0.2.0 must either disable it or report it as a further difference. Until a semantically aligned pair is established, E3 is labeled a *shape-matched native-recipe comparison*; attention-backend matching alone does not make it an equivalent-algorithm framework comparison. Its speed ratio cannot be attributed solely to framework overhead.

A recipe-matched AR systems comparison (E3-AR) is reported below; its two runs were not scored on the external MATH-500 endpoint that Section 8.1 reports for E1, and a controlled time-to-quality comparison remains open.

The full protocol requests median, mean, and p90 iteration time, throughput, phase timing, memory, and utilization. The older report-derived E3 bundle supplies only selected aggregate timings and throughput, whereas E3-AR supplies the post-warm-up per-rollout timing and response-length distribution across 499 measured rollouts (one process-level run per framework). Strong- and weak-scaling sweeps (fixing global or per-GPU batch work, respectively) remain unexecuted extensions contingent on completing a semantically aligned single-node baseline. The 30-rollout E3 protocol does not support a time-to-quality claim; the E2-R quality comparison is still missing and E6 did not complete a threshold-based study.

Table 12: Reported E3 native-recipe systems timings and E5-T deployment-topology timings. All rows use eight allocated GPUs. E3 and E5-T answer different questions; differences in estimators and deployment prevent a pure framework or transport attribution. Aggregate values retain the source table's precision, not an independently established measurement accuracy.

System	Configuration	s/iteration	samples/GPU-h
E3: native recipes, eight GPUs in total			
UniRL	Native flow / SDPA-class	64.2	5381
VeRL-Omni	Native CPS / SDPA	111.3	3104
VeRL-Omni	Native CPS / FA3	112.3	3079
E5-T: deployment regimes, eight GPUs in total			
UniRL	Colocated, 8 shared GPUs	64.223	–
UniRL	Separate, 4 training + 4 rollout GPUs	146.244	–

E3 result and its comparison boundary. The reported mean iteration times are 64.22 s for UniRL and 111.33 s for VeRL-Omni with SDPA (rounded to 64.2 s and 111.3 s in the source summary of Table 12), corresponding to a speedup ratio of approximately 1.73×. The observed SDPA and FA3 times are similar; this is not an equivalence test. The result concerns the complete measured native configurations, including their estimator and LoRA-target differences. The report also records greater short-run reward improvement per iteration for VeRL-Omni, so iteration throughput cannot be converted into a time-to-quality advantage. Neither a parameter-adjusted ratio nor a progress-normalized ratio is presented as an independently validated matched-control result. Generation phase labels are also non-equivalent: VeRL-Omni includes scoring in its generation timer, whereas UniRL separates it. We therefore omit a phase-breakdown figure until a common accounting boundary and the underlying measurements are available. Because the legacy report summaries contain minor discrepancies across replicate notes, Figure 9 plots only the aggregate table values without inferred replicate points, confidence intervals, or statistical significance claims.

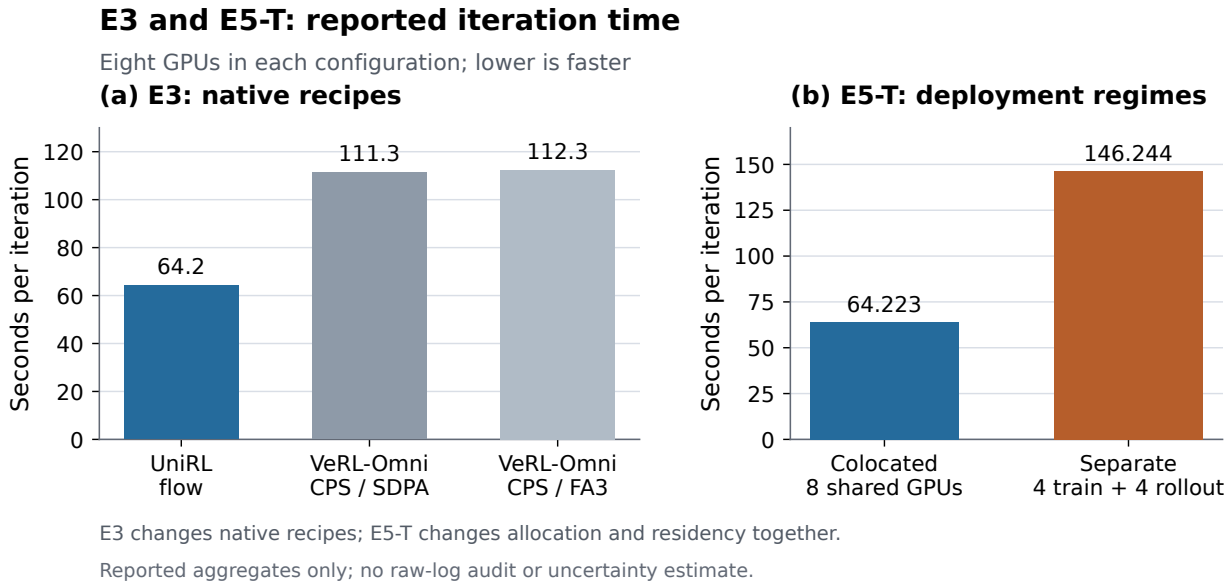


Figure 9: Reported iteration-time summaries. Left: E3 native-recipe comparison on one eight-GPU node. Right: E5-T's separate four-training/four-rollout allocation versus eight colocated GPUs within UniRL. Lower is faster. Panels keep the external recipe comparison separate from the internal deployment comparison; no error bars are inferred from unavailable run-level data.

Recipe-matched autoregressive comparison (E3-AR). We additionally compare UniRL and veRL on a 500-rollout Qwen3-4B-Base (Yang et al., 2025) standard GRPO workload over DAPO-Math-17k (Yu et al., 2025), matching the prespecified RQ1 algorithm. Each framework uses 16 NVIDIA H20 GPUs across two eight-GPU nodes. The matched recipe uses 64 prompts with eight samples per prompt, temperature and top-p 1.0, top-k disabled, natural EOS termination with an 8,192-token response cap, and four optimizer updates per rollout under a 10,240-token per-GPU micro-batch budget. The two runs align the model, dataset and seeded prompt shuffle, tokenizer, verifier, GRPO objective (advantage divided by the group standard deviation, per-sequence token-mean loss (`seq-mean-token-mean`), symmetric clip $\epsilon = 0.2$), rollout log probabilities as the frozen π_{old} anchor, optimizer settings, fp32 master weights with bf16 compute, rollout batch, and update count. UniRL uses eight SGLang TP=2 engines, while veRL uses eight asynchronous vLLM TP=2 engines; these serving stacks are part of the compared end-to-end systems. The recipes do not align the training attention kernel (UniRL uses FlexAttention), reward placement (UniRL scores responses in a separate reward stage, whereas veRL's reward manager runs inside rollout), or realized response lengths, which follow each policy's own training trajectory. We exclude startup rollout 1 as warm-up and report rollouts 2–500 (499 measured iterations).

Table 13: Recipe-matched Qwen3-4B-Base GRPO performance over rollouts 2–500 (one rollout per iteration step). Recipes are matched; realized response lengths and 8,192-token truncation rates are not, and the two runs executed concurrently on separate two-node H20 allocations. Times, response length, and truncation rate are mean ± step-level population standard deviation over 499 rollouts. Token rates divide total generated tokens by total measured time. Step-level deviations describe these runs and are not process-level uncertainty estimates.

System	E2E (s/step)	Generation (s/step)	Training (s/step)	Response tokens	Truncation rate	E2E tok/s	Generation tok/s
UniRL	143.519 ± 28.601	**76.059 ± 15.129**	61.196 ± 13.458	4,197.6 ± 1,005.4	0.307 ± 0.122	**14,975**	**28,257**
veRL	**139.894 ± 30.731**	80.619 ± 16.308	**55.743 ± 14.577**	3,781.8 ± 1,039.1	0.203 ± 0.082	13,841	24,018

AR systems result. Across the 499 measured rollouts, UniRL's mean raw end-to-end iteration time is 2.6% higher than veRL's (143.519 s vs. 139.894 s), and its policy generates 11.0% longer responses on average

(4,197.6 vs. 3,781.8 tokens) with a higher fraction of completions reaching the 8,192-token cap (0.307 vs. 0.203 truncation rate). The median raw end-to-end iteration time is 147.7 s for UniRL and 150.1 s for veRL, and the 90th percentiles are 172.6 s and 169.6 s. Longer responses and more frequent 8,192-token truncation increase per-step token volume (lengthening raw iteration time) but also raise output-token throughput (tok/s) by amortizing per-step and prefill overhead over longer decode sequences, so response length and truncation act as a workload confounder rather than a one-sided handicap. When normalized by total generated output tokens, UniRL records 8.2% higher end-to-end output-token throughput (14,975 vs. 13,841 tok/s) and 17.7% higher generation output-token throughput (28,257 vs. 24,018 tok/s). In raw phase time per step, UniRL is faster in generation by 4.56 s (5.7% lower) but slower in training by 5.45 s (9.8% higher), alongside separate reward (2.27 s) and weight-publication (3.08 s) stages; only on a per-output-token basis is UniRL's generation advantage (+17.7% tok/s) larger than the training difference (where output tokens processed per training second are within 1.1%: 35,119 vs. 34,735 tok/s). The raw end-to-end ordering reverses near rollout 300 as the two sampled response-length distributions cross, while UniRL's token-normalized end-to-end and generation throughput remain higher in all five 100-rollout bands. Per-phase timings rely on each framework's native timers and serve only as diagnostics; end-to-end step time provides the cleanest cross-framework boundary. Two estimator-level differences remain that configuration cannot remove: UniRL divides advantages by the population standard deviation and veRL by the sample standard deviation, making veRL's advantages uniformly $\sqrt{7/8} = 0.935\times$ smaller at eight samples per prompt, and the two advantage epsilons differ by 10^{-8} versus 10^{-6}; both are quantified against their closed forms in the bundle's audit. Finally, because the two runs executed concurrently on different two-node pairs in the cluster and comprise one process-level run per system, this comparison cannot separate framework differences from node-level hardware variance, repeated-run uncertainty, or time-to-quality.

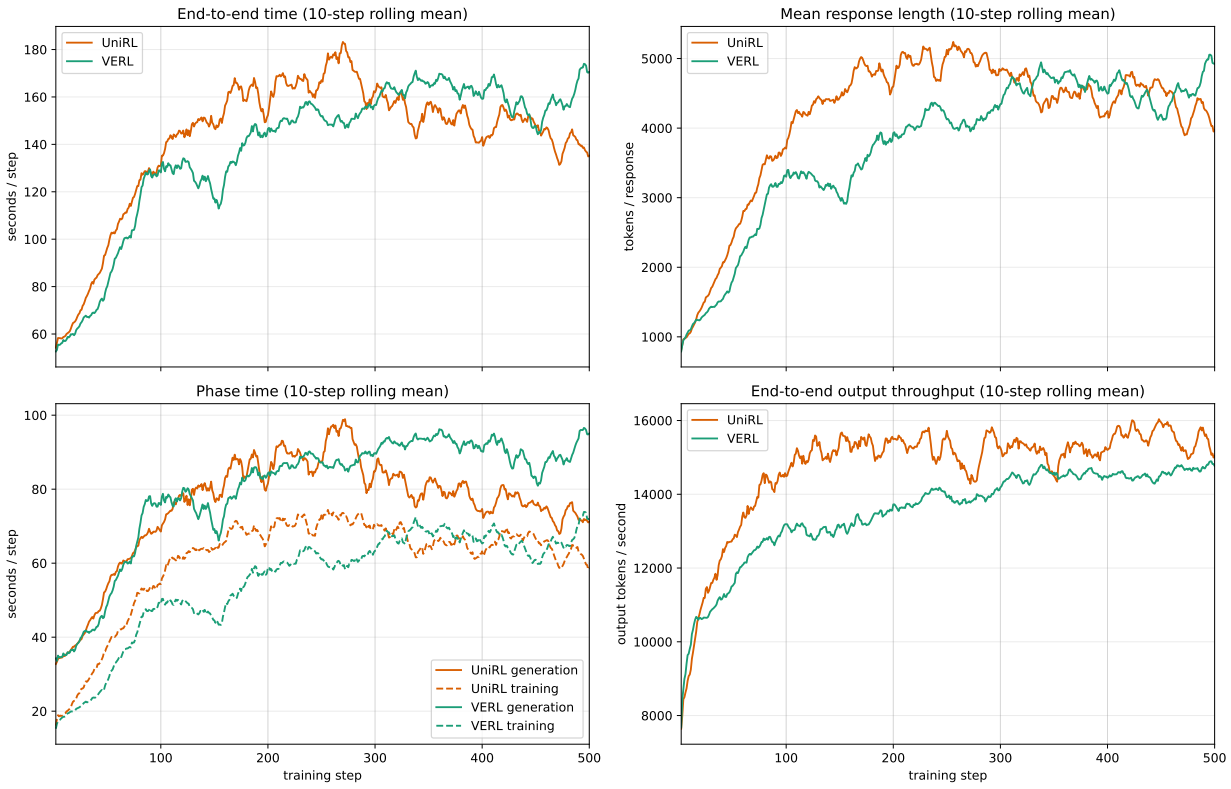


Figure 10: Ten-rollout rolling means for the 500-rollout natural-length Qwen3-4B-Base GRPO comparison. Rollout 1 is excluded as startup warm-up. Response lengths evolve during training and cross near rollout 300, reversing the raw step-time ordering; token-normalized end-to-end throughput remains higher for UniRL throughout the five reported bands.

8.3 RQ3: What does the trajectory abstraction cost and enable?

RQ3 first tests the semantic capability that motivates the IR (E4, comprising synthetic equality verification E4a and three-seed training comparison E4b), and then measures its representation and topology costs (E5, comprising the paired representation microbenchmark E5-R and the deployment-topology comparison E5-T). E4 uses the executable frozen-Qwen3-0.6B rewrite $\rightarrow$ SD3.5-Medium path: eight original prompts, four rewrites per prompt, and eight images per rewrite, or 256 image descendants per rollout; only diffusion trains. This is a separate recipe from E2: 512×512, ten steps, one sampled SDE index among 0–8, $\eta = 0.7$, LoRA 16/32, learning rate 3×10^{-4}, weight decay zero, clip 10^{-4}, replay anchor, two updates, and micro-batch size one. The primary condition, which we refer to as *root scope* (`diffusion_group_scope=prompt`), computes group-relative advantages across all 32 image descendants of each original root prompt, whereas *rewrite scope* (`diffusion_group_scope=rewrite`, the trainer default) normalizes across the eight images generated under each individual rewrite. Because the two stock prompt-enhancement recipes are not otherwise matched—the stock prompt-scope file specifies eight images per rewrite and a 512-token rewrite limit, whereas the stock rewrite-scope file specifies four images and 256 tokens—both arms use the prompt-scope recipe with only the grouping key varied, as verified in their resolved configurations (both stock recipes also enable `shuffle_updates` at the audited release). Both conditions serialize redacted root, layer, and parent IDs, tensor shapes, rewards, advantages, replay indices, and group labels; because the synchronous prompt-enhancement trainer does not increment behavior-policy versions, serialized layers carry no behavior-version provenance in this workflow. Invalid lineage remains an E0 fail-fast check rather than a corrupted training baseline.

For image $x_{p,j,k}$, the two group keys are $g_{\text{root}} = p$ and $g_{\text{rewrite}} = (p, j)$. The semantic verification test (E4a) requires an explicit ID-joined flat-row reference to reproduce UniRL's group memberships, advantages, and learner inputs on identical recorded rewards and replay tensors, including across root-shard split/gather round trips. Passing that recorded-trajectory control establishes that the IR implements the intended grouping semantics on real rollout data, though it does not by itself imply higher downstream image quality. Conversely, because the 300-rollout, three-seed training comparison (E4b) alters the advantage-normalization population (32 versus eight images), any quality difference reflects the choice of grouping objective rather than the overhead of the IR representation. The current PickScore request is conditioned by the rewritten text (the default `prompt_source: generation`; scoring against the root prompt with `prompt_source: original` is available but was not used in E4), so a claim about preserving the user's original intent additionally requires lineage-keyed evaluation of every image against its root prompt. We therefore report both rewrite-conditioned training reward and frozen HPSv3/ImageReward evaluations joined to each root prompt. Evaluation generates held-out images at 512×512, 40 steps, and guidance 1 with the base and every 50-rollout diffusion checkpoint, using one fixed manifest of four frozen rewrites per root and eight image seeds per rewrite for both conditions. Worker/adapter RNG initialization, deterministic train-side AR sampling, real trace dumping, and this held-out generation/evaluation path are E0 prerequisites for a multi-seed E4 result. If the two correct groupings tie, the experiment still establishes semantic expressiveness but not a quality benefit.

E4 grouping semantics: synthetic fixtures and recorded rollouts. On synthetic fixtures, E4a passes all 24/24 checks under both grouping scopes in bit-exact agreement with an explicit ID-joined reference. The corrected E4b training campaign (denoted E4 v2 below) extends this verification to recorded rollouts: across the first two training rollouts of all six runs, the execution record confirms that UniRL's group memberships and advantages match those computed by an independent ID join over the saved sample IDs and rewards (forming eight groups of 32 images under root scope and 32 groups of eight under rewrite scope). While this verifies group membership and advantage calculation on live rollout data, replay tensors, learner inputs, and root-shard round trips remain verified only on the synthetic E4a fixtures, so the full recorded-trajectory control is only partially satisfied; moreover, the raw outputs of the recorded-rollout check remain on cluster storage rather than in the repository bundle.

Table 14: E4 v2 on held-out root prompts: root-conditioned HPSv3 at checkpoint 300 under each grouping scope, one column per training seed. The last row is the registered contrast, root scope minus rewrite scope within each training-seed pair, with a 95% t interval over the three pairs.

	Seed 11	Seed 22	Seed 33	Mean	95% interval
Frozen base (no training)		2.98			–
Root scope	6.06	8.67	8.24	7.66	–
Rewrite scope	9.17	8.19	8.63	8.66	–
Root scope minus rewrite scope	−3.11	+0.48	−0.39	−1.01	[−5.65, +3.64]

E4b quality comparison: inconclusive. For the three-seed training comparison (E4b), the corrected campaign (E4 v2) evaluates root scope against rewrite scope using a fixed rewrite prompt template under a statistical analysis preregistered before launch (Table 14). E4 v2 supersedes an initial E4b campaign in which the rewriter lacked its system instruction and output-extraction filter (causing two of the six runs to finish below the frozen base and the two scopes to train on non-identical rewrites); that initial campaign was likewise inconclusive (95% t interval [−14.23, +9.97]), and its outputs remain archived in the bundle. In E4 v2, the two arms differ in only a single configuration line (`diffusion_group_scope`), share identical training rewrites and random seeds within each seed pair, and verify that the frozen Qwen3-0.6B rewriter weights remain unchanged throughout training. All six runs improve over the frozen base (2.98) on root-conditioned HPSv3 at rollout 300, reaching 6.06 to 9.17, and none triggers the preregistered collapse guard. The primary preregistered contrast—root scope minus rewrite scope on root-conditioned HPSv3 at rollout 300—is −3.11, +0.48, and −0.39 across the three seed pairs, with a mean difference of −1.01 and a 95% t interval of [−5.65, +3.64], which is inconclusive across three seed pairs. More broadly, none of the 24 seed-level confidence intervals across saved checkpoints and evaluation metrics (root- and rewrite-conditioned scorers) excludes zero. This null result does not establish equivalence or a zero true effect: the interval half-width (4.65) is comparable to the overall training gains over the base model (3.1 to 6.2). The held-out root prompts are drawn from the stock PickScore test split, 32 of which also appear in the training split after text normalization; both splits were used as shipped. Overall, E4 establishes that the trajectory IR expresses both lineage-defined grouping objectives, computes their group advantages in agreement with an explicit ID join on recorded rollouts, and supports end-to-end training under both configurations, without demonstrating a downstream quality advantage for either scope.

Paired representation cost (E5-R) and deployment topology (E5-T). The intended E5-R microbenchmarks separate recursive IR operations and tensor transport from model compute, varying root count, branch factor, tensor payload, token/media lengths, and layer depth. Required outputs include structural-operation time, metadata bytes, dense bytes avoided at the driver, transfer time, and reassembly correctness hashes.

The broader E5 protocol compares the colocated store, per-GPU store, and TransferQueue path where hardware permits. The current harness does not yet implement the paired E5-R representation measurement: whole-tree sharding versus explicit ID-joined flat rows on the same recorded payload, device, transport, and operations. This paired representation benchmark holds model computation, device placement, decoded payload tensors, group assignments, and replay inputs fixed; driver resident set size (RSS), bytes transferred, structural operation latency, and share of total iteration time then isolate the cost of preserving tree structure. Semantic errors are caught in E0 and are never timed as valid work.

The minimum topology slice (E5-T) reuses E3's colocated SD3.5-Medium UniRL row and compares it against the same workload under `sd3_vllmomni_lora_separate.yaml`, where four GPUs train and four GPUs run rollout within the same eight-GPU allocation. Because that stock separate-slab recipe differs from E3's aligned settings, the E3 overrides must be applied and archived with the separate arm. E5-T compares two complete deployment regimes rather than an isolated transport primitive: training DP width, memory residency, and local versus cross-slab LoRA publication change together and require a common phase breakdown before attributing their costs. Broader same-/cross-GPU and cross-node backend measurements remain E5 extensions. The reported E5-T iteration times are 64.223 seconds for colocated deployment and

146.244 seconds for the separate allocation, approximately 2.277 times longer (Figure 9). The separate UniRL configuration is also slower than the colocated VeRL-Omni E3 configuration. The experiment report attributes much of the increase to reward processing, but without exported phase records we retain this as a reported diagnosis rather than an isolated causal attribution. Neither this deployment penalty nor the CPU timings below establish bounded representation overhead. To document the $N_{\text{mod}} + N_{\text{alg}}$ composition in Section 3.6, the E4 path adds only workflow-specific bundle and trainer wiring while reusing the core trajectory IR, scheduler, transport, and credit modules unchanged.

Preliminary executable evidence. We ran a source-importing regression harness against a clean UniRL checkout at `fd344ad`; it does not mock the trajectory classes. All ten checks passed: cross-stage lineage construction, exact tree split/concat, whole-tree select/slice, leaf-to-root reward propagation and group advantages, rejection of malformed lineage and primitive modality/batch violations, variable-length packed-text round trips, and lazy `TensorRef` selection/materialization including bounds checking. Table 15 reports a deliberately narrow timing slice from the same run. The synthetic workload has four AR children per root, one observation per AR child, two diffusion children per observation, and a four-value image payload per leaf.

Table 15: Preliminary single-process CPU trajectory-IR timings (microseconds). Python 3.12.13 and PyTorch 2.11.0; one thread; ten warm-up and 100 measured repetitions. These are regression measurements, not distributed or GPU throughput.

Roots	Leaf rows	Operation	Median	p90
8	64	`sample.split_concat`	120.8	142.6
8	64	`sample.select_half`	556.8	616.7
8	64	`tensorref.select_strided`	12.3	13.0
32	256	`sample.split_concat`	291.2	329.6
32	256	`sample.select_half`	2231.5	3126.8
32	256	`tensorref.select_strided`	47.0	52.4
128	1024	`sample.split_concat`	1435.5	2231.7
128	1024	`sample.select_half`	8152.4	8458.7
128	1024	`tensorref.select_strided`	202.2	226.4

The current `Sample.select` path forms tree-complete root shards before gathering them, so its cost grows with the full input tree rather than only the selected rows. Strided `TensorRef` selection creates many metadata spans while contiguous selection coalesces them; no dense tensor is copied until materialization. These observations motivate, but do not replace, the planned GPU transport and end-to-end ablations. The manifest, 100 per-cell raw trials, structured check outcomes, generated table rows, and SHA-256 checksums are archived under `artifacts/trajectory_ir/cpu_fd344ad_macos_arm64/`.

8.4 RQ4: Exploratory bounded-staleness observations

The prespecified E6 study uses the E1 Qwen3-4B-Base GRPO workload on DAPO-Math-17k across 32 GPUs (64 prompts × eight samples and four optimizer updates per rollout). Its grid separates three factors: (i) device allocation, comparing colocated synchronous execution on all 32 GPUs (`S0`) against disaggregated 16-training/16-rollout execution with one batch in flight, weight publication after every batch, and zero batch-staleness allowance ($b = 0$, so policy lag $\ell = 0$; `D0`); (ii) the onset of generation–training overlap, comparing `D0` against `A1` (two batches in flight, publication interval 1, staleness allowance $b = 1$); and (iii) wider staleness allowances and publication intervals (`A2`: in-flight 2, interval 1, $b = 2$; `A3`: in-flight 2, interval 2, $b = 2$; `A4`: in-flight 2, interval 2, $b = 4$). The formal grid sweep was canceled on 2026-09-21 after preliminary runs; only single-seed capacity measurements (with evaluation disabled) and a separate quality-target pilot on `D0` were completed. In the single-seed capacity runs, reported per-rollout durations were 82.34 s for `S0`, 103.07 s for zero-overlap disaggregated `D0`, and 56.48 s (`A1`), 55.19 s (`A2`), 33.47 s (`A3`), and 39.54 s (`A4`, pooled over two replicates) once overlap was enabled. Because those preliminary timings were collected at one seed with evaluation disabled—and before aligning five stock learner configuration differences between the synchronous and asynchronous recipes—they serve only as planning observations rather than a controlled speedup or equal-quality comparison.

The zero-lag quality-target pilot did not pass. Before running the overlapped grid for quality, the protocol required registering a target MATH-500 threshold from the disaggregated zero-overlap baseline `DO`. In that pilot, `DO`'s reported MATH-500 avg@4 decreased from 0.798 at rollout 150 to 0.773 at rollout 300, contrary to the prespecified requirement that the later checkpoint improve, and training reward declined over the same window. No quality threshold was registered and no time-to-quality result is reported. Because `DO` enforces one batch in flight, weight publication after every batch, and zero allowed policy lag ($b = 0 \Rightarrow \ell = 0$, so generation and training strictly alternate without staleness; Section 4.5), its degradation reflects the training horizon and learner dynamics of the zero-lag baseline rather than policy staleness. Because no configuration with $b > 0$ ($\ell > 0$) or generation–training overlap was evaluated for quality before cancellation, E6 provides no empirical evidence on how bounded staleness affects model quality. Two checkpoints are also insufficient to locate a precise turning point.

Evaluation variability and requirements for a restart. Repeated evaluation of a single checkpoint exhibited noticeable prompt-level avg@4 variance despite a small aggregate difference. That observation is specific to the sampled checkpoint and decoding protocol; it is not a universal noise floor or a substitute for training-seed replication. A renewed study would first audit scorer behavior, request-level sampling, and repeat-evaluation variability; match the synchronous/asynchronous estimator and learner settings; and record complete driver time and buffer admission, rejection, discard, retry, and completion accounting. The trainer's internal `perf/step_time_s` metric alone omits outer-loop weight publication, evaluation, and checkpoint work. Only after establishing a suitable learning budget and a separately piloted target should a replicated formal grid support a lag–quality–time conclusion.

8.5 Fairness and statistical protocol

Effective work is matched before wall-clock time: prompts, samples per prompt, decoded resolution or maximum tokens, rollout steps, replayed steps/tokens, micro-batching semantics, and optimizer updates. Primary full-system metrics count all allocated training, rollout, and reward resources on both sides. Reward placement and concurrency must be disclosed; a reward-excluded number is supplementary and carries a separate service-cost report. We prespecify warm-up removal, use at least 20 steady-state iterations for throughput summaries, and report the full distribution rather than a single minimum. Training summaries use the training seed as the primary statistical unit. Across E1, E2, and E4, individual seed trajectories or endpoint scores are shown in Table 10 and Figure 5 (E1), Figure 6 (E2), and Table 14 (E4), while seed-level 95% t intervals (with two degrees of freedom for three seeds, and thus wide by construction) accompany the three-seed endpoint changes in Table 11 and the paired scope contrast in Table 14. Bootstrap intervals over prompts or images are labeled separately and do not substitute for cross-seed uncertainty. The complete systems release must show all process repetitions and the post-warm-up per-rollout distribution; uncertainty of a speed ratio must use repetitions, not correlated rollouts, as its independent units. E3-AR releases its 499-rollout post-warm-up distribution, but has only one process-level run per framework; the older report-derived E3 summaries provide neither the complete per-rollout distribution nor independently revalidated repetitions. The final checkpoint and named primary endpoint are fixed in advance; the auxiliary metrics diagnose failure modes and are not alternative endpoints chosen after seeing results. Any future E6 time-to-quality study must use MATH-500 and a target prespecified from a separate pilot before the formal grid, bracket first observed crossings by evaluation times, and censor runs that never cross. The canceled study produced no such result. Elapsed time spans initialization, training, publication, checkpointing, and evaluation; steady-state throughput is reported separately. All published cells require complete scoring or an explicit missingness analysis; silently averaging only successful evaluator responses is inadmissible.

9 Related Work

LLM post-training systems. HybridFlow/veRL combines single- and multi-controller execution and introduces hierarchical APIs for mapping RLHF dataflows to devices (Sheng et al., 2024). OpenRLHF uses Ray, vLLM, and DeepSpeed to schedule the models in an RLHF workflow (Hu et al., 2024). ROLL emphasizes a single-controller abstraction, parallel workers, sample lifecycle scheduling, and flexible device mapping (Wang et al., 2025). NeMo-RL provides scalable training and generation backends and multimodal

model support (NVIDIA Corporation, 2025). Group-relative recipes in these systems (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025) typically record one level of branching—responses grouped under their prompt by a prompt identifier or by contiguous row order—whereas UniRL generalizes that group key to multi-level ancestry encoded in trajectory-node identifiers (`sample_id` and `parent_id` within a `Sample`). These systems already drive distinct generation and training backends from one control loop and colocate or separate those backends as a placement choice; UniRL retains that execution split while replacing flat row batches with lineage-preserving `Sample` trees as the scheduled unit. Scheduling and DP sharding move complete root groups rather than rows, and each generated layer carries the replay segment for its own AR or diffusion stage (Section 3). A flat-row batch can recover the same group memberships by joining on retained parent keys (Section 2); UniRL differs in that sharding, credit assignment, and cross-stage conditioning consume that ancestry in place, so changing the execution path does not require rebuilding the join. RQ3 specifies the corresponding control: an ID-joined flat-row reference over recorded rewards and replay tensors (Section 8.3). To date, that reference check covers group membership and advantages on recorded training rollouts (E4 v2), whereas replay tensors, learner inputs, and root-shard round trips are verified only on synthetic fixtures (E4a); the paired representation cost of carrying tree structure (E5-R) remains unmeasured beyond single-process CPU regression timings.

Asynchronous post-training. AReaL fully decouples generation and training and controls staleness to improve language-reasoning throughput (Fu et al., 2025). Relax uses service-level separation, a TransferQueue data bus, and configurable staleness for omni-modal post-training (Zhang et al., 2026). slime supports persistent asynchronous rollout generation around a training/rollout/data-buffer loop (Zhu et al., 2025). Within the asynchronous design space explored by these systems, UniRL adopts a conservative operating point in which weight publication remains a synchronization barrier and policy lag is bounded in optimizer-update units (Section 4.5). Each generated layer records its behavior-policy version in the trajectory IR, while buffer admission, root-complete lag filtering, and publication barriers are coordinated by a shared asynchronous rollout manager across `AsyncARTrainer` and `AsyncDiffusionTrainer`. The filter keeps or drops whole root groups so that group-relative baselines stay complete, and no training batch spans behavior versions; this is stricter than AReaL, whose decoupled objective admits trajectories generated under several behavior versions (Fu et al., 2025). RQ4 reports only single-seed capacity measurements and a quality-target pilot that did not pass; it supports no throughput or time-to-quality comparison with these systems (Section 8.4).

Multimodal and diffusion post-training systems. VeRL-Omni extends veRL toward diffusion, unified, and omni-modality models and integrates vLLM-Omni rollout (Huang et al., 2026). Relax is explicitly omni-native (Zhang et al., 2026). Rather than modality coverage alone, what distinguishes UniRL from these systems is how ancestry and typed replay segments survive execution: an AR stage and a diffusion stage are represented as successive layers of a single `Sample` (Section 3), and tensor references transport their dense payloads without altering that lineage (Section 4). Because model bundles and `StageAlgorithm` implementations interact only through typed stage conditions, replay segments, and replayed sufficient statistics, supporting N_{mod} model bundles and N_{alg} shared algorithms requires $N_{\text{mod}} + N_{\text{alg}}$ rather than $N_{\text{mod}} \times N_{\text{alg}}$ integrations (Section 3.6). E4 exercises this boundary by coupling a frozen AR prompt-rewrite stage to a trained diffusion stage (Section 8.3). Its lineage check verifies group membership and advantages on recorded training rollouts (E4 v2) and replay-tensor propagation on synthetic fixtures (E4a), while its three-seed quality comparison between root-level and rewrite-level grouping is inconclusive. A static module inventory can document which trajectory, transport, and credit components this pipeline reuses, though it does not measure human engineering effort.

Diffusion RL algorithms. DDPO casts denoising as a multi-step decision process (Black et al., 2023). Flow-GRPO converts flow sampling to an SDE and applies group-relative optimization (Liu et al., 2025); DanceGRPO generalizes related ideas across visual generation tasks (Xue et al., 2025); DiffusionNFT instead optimizes a forward-process objective from reward-ranked generations (Zheng et al., 2025). UniRL implements Flow-GRPO-style and DiffusionNFT objectives behind its `StageAlgorithm` interface without claiming either objective as an algorithmic contribution (Section 5). Instead, the interface keeps their distinct sufficient statistics explicit: Flow-GRPO-style updates replay recorded reverse-SDE transitions, whereas

DiffusionNFT evaluates forward-process noising losses on terminal generated latents, while both share the same `TrainStack` for micro-batching and optimizer state.

10 Limitations and Open Evidence

Integration cost is per architecture. Adding an architecture requires one bundle with its stages, conditions, and rollout adapters; this cost is paid once per model rather than once per model–algorithm pair. Architecture-specific details such as tokenizers, noise schedulers, latent layouts, and LoRA targets stay inside that bundle and its adapters rather than being abstracted by the trajectory type. The ordered-layer representation targets trajectories that grow by forking and observation; arbitrary ragged forests are outside its current scope.

Topology is configured, not optimized. Placement scopes make device allocation explicit but do not automatically search for an optimal resource split across roles, which depends on model memory footprint, reward-evaluation latency, rollout length variance, interconnect bandwidth, and weight-publication volume.

Asynchronous execution is bounded and partially specialized. Weight publication is a barrier, and current AR and diffusion paths impose different admissible in-flight structure. Fault isolation is limited by the shared Ray runtime and role lifecycle; the system does not yet claim service-level recovery comparable to dedicated service architectures.

Case-study scope. The reward-serving and rollout-fidelity case studies (Sections 6 and 7) evaluate targeted mechanisms outside the E0–E6 program on a single BAGEL image-editing workload with small rollout counts per arm. In Section 6.4, the image-editing wall-clock durations are rescaled estimates that combine measured reward-side traces with the generation speedup from restored source conditioning; in Section 6.5 and Section 7, the MPS colocation runtime, token-layout check, per-group canvas, sibling packing, and cost-aware load balancer are evaluated from open UniRL pull requests (#492, #547–#550) or an experimental branch rather than the v0.2.0 release commit.

The empirical case is incomplete. The GPU experiments support only qualified, workload-specific observations: (i) diffusion post-training (E2) improves held-out automatic scores while reducing within-prompt diversity; (ii) autoregressive training (E1) exhibits early gains followed by late-stage degradation under the prespecified GRPO recipe, whereas a post-hoc DRPO recipe improves over the base model as graded (with one seed whose final-checkpoint answers largely fail to terminate); (iii) the composed rewrite-to-image pipeline (E4 v2) computes both lineage-defined groupings accurately and trains above the base model; and (iv) the systems experiments measure native-recipe iteration time (E3), colocated-versus-separate deployment sensitivity (E5-T), and single-run output-token throughput in a recipe-matched AR comparison (E3-AR). Because E3-AR comprises a single process-level run per framework without external checkpoint evaluation, it does not establish run-to-run variance, training-quality parity, or time-to-quality. More broadly, the current evidence does not establish mathematically aligned reference reproduction (E2-R), endpoint improvement under the prespecified AR recipe (E1), a quality advantage from root-level over rewrite-level grouping (E4 v2), bounded IR representation overhead (E5-R), multi-node scaling, or a time-to-quality benefit from asynchronous execution (E6). The E3, E5-T, and E6 summaries remain derived from legacy experiment reports: their raw GPU logs are unavailable, and they still lack complete run manifests, reconciled replicate provenance, and independent regeneration. The E0, E1, E2, and E4 tables, figures, and quoted statistics are generated directly from exported per-run evaluation and rollout records in the repository, but their underlying completions, generated images, full runtime logs, and checkpoint weights remain on cluster storage outside the released bundle. Furthermore, while E4 v2 validates group membership and advantage computation on recorded rollouts, it neither archives the validation outputs in the bundle nor checks replay tensors on recorded payloads, and its three-seed quality contrast remains statistically inconclusive. The late-stage GRPO degradation also remains unexplained: no matched external reference implementation was evaluated on the E1 protocol, and the DRPO configuration differs from the GRPO configuration across five simultaneous settings, preventing attribution to the policy loss alone. Moreover, the final-checkpoint

rank ordering between DRPO and GRPO holds only under default `math_verify` grading and disappears if completions that reach the 16,384-token generation limit are scored as incorrect.

11 Conclusion

UniRL formulates post-training around a single trajectory representation that persists across generative model families and execution topologies. Ancestry, decoded multimodal payloads, stage-specific replay segments, rewards, advantages, and behavior-policy versions reside in one trajectory IR, while rollout, reward evaluation, and training operate as logical roles connected by explicit placement, transport, and weight-publication contracts. Two case studies on instruction-guided image editing illustrate how these boundaries operate in practice: micro-batched generation–reward overlap hides remote judge latency when the judge has spare capacity, while structural token-layout checks and sibling request packing across the rollout-engine boundary recover conditioning fidelity and reduce generation time. Across the main evaluation program, our experiments show (i) held-out diffusion score gains accompanied by reduced within-prompt diversity (E2), (ii) early autoregressive learning followed by late-stage degradation below the base model under the prespecified GRPO recipe, alongside positive final-checkpoint gains under a post-hoc DRPO recipe across all three seeds as graded (E1), (iii) exact agreement with an ID-joined reference on recorded rollouts and positive training gains under both lineage-defined groupings in the composed rewrite-to-image workflow (E4 v2), and (iv) lower native-recipe diffusion iteration time than VeRL-Omni (E3), a substantial iteration-time penalty for disaggregated diffusion deployment (E5-T), and 8.2% higher end-to-end output-token throughput than veRL in a single recipe-matched Qwen3-4B-Base GRPO run (E3-AR). That AR systems result is a workload-specific throughput observation rather than a repeated-run or time-to-quality conclusion. The full trajectory-centric claim remains open until replay-level lineage validation on recorded payloads and matched representation-cost measurements complement those observations, and bounded staleness remains an exploratory operating-region question with no demonstrated time-to-quality benefit in the canceled study.

Reproducibility statement

The core architectural characterization in Sections 3–5 refers to UniRL release v0.2.0, commit `fd344ad6036a0ba7be9900a9d0cf78ebf59fa738` (2026-10-08). The companion report repository records the claim-to-evidence plan (`PAPER_PLAN.md`), the cluster execution protocol and configuration-parity gates (`EXPERIMENT_RUNBOOK.md`), the operational handoff commands (`GPU_HANDOFF_README.md`), and the result audit (`EXPERIMENT_RESULTS.md`). The legacy report bundle `artifacts/reported_results/2026-09-22/` (integrating the 2026-09-22 experiment summary and companion report pull request #2 at commit `ecce5097`) stores compact source snapshots, source-level caveats, reported numerical inputs, and generated tables and figures for E3, E5-T, and E6. These reproduce the transcription and presentation of the legacy reports rather than the original raw GPU logs. The reward-serving and rollout-fidelity case studies (Sections 6 and 7) archive their per-rollout records, rescaling models, and table/figure generators under `artifacts/reported_results/2026-09-30-reward-service/` (UniRL commit `651490e` and UniRL pull request #492 at `8bb4119`) and `artifacts/reported_results/2026-10-04-rollout-fidelity/` (UniRL pull requests #547–#550). The separately rerun AR systems bundle in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/` pins UniRL commit `6148a5d`, the veRL reference and compatibility commits, both resolved recipes, the recipe and effective-work audit, run identities, 499 post-warm-up records per framework, phase and band summaries, and the plotting script. Commit `6148a5d` is on a pre-release branch: it descends from an earlier audit commit and carries four rollout-performance and decoding commits not included in v0.2.0, so E3-AR measures that branch rather than the audited v0.2.0 tag. Its UniRL run is W&B run `2x4tdjwa`; its veRL run is `grpo-verl-natural500`, whose per-rollout records come from the trainer's console metrics. The HunyuanImage-3.0 and MiniMax-H3 bundles in `artifacts/reported_results/2026-09-30-hunyuanimage3/` and `artifacts/reported_results/2026-09-30-minimax-h3/` store the resolved recipes, W&B run identities, direct per-rollout history exports, and rendered learning curves. The dated bundle `artifacts/reported_results/2026-10-08-e0-e1-e2-e4/` stores data exported from the E0, E1, E2, and E4 runs, with a generator that reproduces their tables, figures, and quoted numbers; its README lists

run identities, source commits, and the evidence tier of each part. All graphics are committed PDF assets included by relative paths, so Overleaf does not require Python or shell escape to build the manuscript. The preliminary RQ3 CPU evidence is regenerated by `experiments/trajectory_ir/run_cpu_evidence.py`; its manifest pins the source state, command, environment, workload, warm-up, repetitions, and timer.

References

Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. 2023. URL https://arxiv.org/abs/2305.13301.

Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. Emerging properties in unified multimodal pretraining. 2025. URL https://arxiv.org/abs/2505.14683.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. 2025. URL https://arxiv.org/abs/2505.24298.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. 2024. URL https://arxiv.org/abs/2405.11143.

Yongxiang Huang, Cheung Kawai, Jingan Zhou, Yingshu Chen, openYuanrong Team, and Xibin Wu. VeRL-Omni: Easy, fast, and stable RL training for diffusion and omni-modality models. GitHub repository, 2026. URL https://github.com/verl-project/verl-omni.

Amita Kamath, Kai-Wei Chang, Ranjay Krishna, Luke Zettlemoyer, Yushi Hu, and Marjan Ghazvininejad. GenEval 2: Addressing benchmark drift in text-to-image evaluation. 2025. URL https://arxiv.org/abs/2512.16853.

Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2305.01569.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. 2025. URL https://arxiv.org/abs/2505.05470.

Xin Luo, Jiahao Wang, Chenyuan Wu, Shitao Xiao, Xiyan Jiang, Defu Lian, Jiajun Zhang, Dong Liu, and Zheng Liu. EditScore: Unlocking online RL for image editing via high-fidelity reward modeling. 2025. URL https://arxiv.org/abs/2509.23909.

Qianli Ma, Yaowei Zheng, Zhelun Shi, Zhongkai Zhao, Bin Jia, Ziyue Huang, Zhiqi Lin, Youjie Li, Jiacheng Yang, Yanghua Peng, Zhi Zhang, and Xin Liu. VeOmni: Scaling any modality model training with model-centric distributed recipe zoo. 2025a. URL https://arxiv.org/abs/2508.02317.

Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards wide-spectrum human preference score. 2025b. URL https://arxiv.org/abs/2508.03789.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

NVIDIA Corporation. NVIDIA multi-process service. Documentation. URL https://docs.nvidia.com/deploy/mps/latest/.

NVIDIA Corporation. NeMo RL: A scalable and efficient post-training library. GitHub repository, 2025. URL https://github.com/NVIDIA-NeMo/RL.

Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. 2024. URL https://arxiv.org/abs/2407.00079.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021. URL https://arxiv.org/abs/2103.00020.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL https://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. 2024. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. 2024. URL https://arxiv.org/abs/2409.19256.

Stability AI. Stable diffusion 3.5 medium model card. Hugging Face model repository, 2024. URL https://huggingface.co/stabilityai/stable-diffusion-3.5-medium.

Weixun Wang et al. ROLL: An efficient and user-friendly scaling library for reinforcement learning with large language models. 2025. URL https://arxiv.org/abs/2506.06122.

Keming Wu, Sicong Jiang, Max Ku, Ping Nie, Minghao Liu, and Wenhu Chen. EditReward: A human-aligned reward model for instruction-guided image editing. 2025. URL https://arxiv.org/abs/2509.26346.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. 2023. URL https://arxiv.org/abs/2306.09341.

Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2304.05977.

Zeyue Xue, Jie Wu, Yu Gao, Fangyuan Kong, Lingting Zhu, Mengzhao Chen, Zhiheng Liu, Wei Liu, Qiushan Guo, Weilin Huang, and Ping Luo. DanceGRPO: Unleashing GRPO on visual generation. 2025. URL https://arxiv.org/abs/2505.07818.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. 2025. URL https://arxiv.org/abs/2505.09388.

Jiarui Yao, Xiangxin Zhou, Penghui Qi, Wee Sun Lee, Liefeng Bo, and Tianyu Pang. Rethinking the divergence regularization in LLM RL. 2026. URL https://arxiv.org/abs/2606.09821.

Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, and Hongsheng Liu. vLLM-Omni: Fully disaggregated serving for any-to-any multimodal models. 2026. URL https://arxiv.org/abs/2602.02204.

Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. 2025. URL https://arxiv.org/abs/2503.14476.

Liujie Zhang, Benzhe Ning, Rui Yang, Xiaoyan Yu, Jiaxing Li, Lumeng Wu, Jia Liu, Minghao Li, Weihang Chen, Weiqi Hu, and Lei Zhang. Relax: An asynchronous reinforcement learning engine for omni-modal post-training at scale. 2026. URL https://arxiv.org/abs/2604.11554.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://arxiv.org/abs/1801.03924.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Bali, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. 2023. URL https://arxiv.org/abs/2304.11277.

Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. 2025. URL https://arxiv.org/abs/2509.16117.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, and Joseph E. Gonzalez. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2312.07104.

Zilin Zhu, Chengxing Xie, Xin Lv, and contributors. slime: An LLM post-training framework for RL scaling. GitHub repository, 2025. URL https://github.com/THUDM/slime.

A Claim-to-source audit

Table 16 records the executable source used to support mechanism descriptions. It does not prove performance or successful execution for every configuration.

Table 16: Primary code evidence at commit `fd344ad` (v0.2.0).

Mechanism	Primary source symbols
Trajectory IR	`unirl/types/sample.py`: `Part`, `Sample`; `unirl/types/primitives.py`; `unirl/types/segments/`
Batch algebra	`unirl/distributed/tensor/batch.py`: `FieldKind`, `Batch`
Tensor references	`unirl/distributed/tensor/ref.py`: `TensorSpan`, `TensorRef`; `unirl/distributed/tensor/transport.py`: `TensorTransport`
Roles and placement	`unirl/distributed/group/remote.py`: `Remote`; `device_pool.py`, `placement.py`, `handle.py`, `dispatch.py`
Rollout boundary	`unirl/rollout/engine/base.py`: `BaseRolloutEngine`; concrete engines under `unirl/rollout/engine/`, including `vllm/`; `unirl/reward/stack.py` for overlapped scoring
Training boundary	`unirl/algorithms/base.py`: `StageAlgorithm`; `unirl/train/stack/base.py`: `TrainStack`; `anchor.py`, `planner/update.py`
Sync trainers	`unirl/trainer/ar.py`: `ARTrainer`; `unirl/trainer/diffusion.py`: `DiffusionTrainer`; `pe.py`; `residency.py`
Async and agentic	`unirl/trainer/async_rollout.py`, `async_ar.py`, `async_diffusion.py`, `agentic.py`; `unirl/rollout/manager/`
Weight publication	`unirl/distributed/weight_sync/`, including `full/ckpt_engine_ipc.py`; engine-specific receivers under `unirl/rollout/engine/`
Instrumentation	`unirl/utils/profiling.py`, `memory_monitor.py`, `wandb_logger.py`

B Colocated residency orders

Table 17 lists the orders that the trainers at the audited release use on a colocated slab. Each satisfies the two invariants of Section 4.3: any due weight publication completes before generation begins, and the engine is put back to sleep before offloaded training state is restored for the training update.

Table 17: Colocated residency orders. "Offload" moves training state to host memory; "sleep" releases the engine's weight and cache memory, and "wake" restores it.

Order	Applies when	Before generation	After generation
Default	AR, agentic, and prompt-enhancement trainers, unless a row below applies	Wake the engine, publish weights if due, offload training state if enabled	Sleep the engine, then restore training state
Split wake	AR; publication due, offload enabled, engine can restore weight memory separately	Wake weight memory only, publish weights, offload training state, complete the wake	As default
Split wake, vLLM weight publication	As above, publishing through vLLM's native weight-transfer engine	Offload optimizer state, wake weight memory, publish weights, offload the model shard, complete the wake	As default
Offload without publication	AR; offload enabled, no publication due	Offload training state, then wake the engine	As default
Extract, then publish	AR with the engine pinned to one designated GPU of the slab; unified-model trainer with separate AR and diffusion engines	Extract weights if due, offload training state if enabled, wake the engine(s), publish the extracted weights	As default; a pinned AR engine with offload disabled stays awake
Per-role residency	Diffusion trainers	Extract the adapter first if training will be offloaded; offload every non-resident role, wake the engine, publish weights if due	Sleep the engine unless the rollout role is resident; restore training state before the optimizer step

C Required run artifact schema

Every result row must resolve to a run directory containing:

- source and baseline commit hashes, submodule hashes, and dirty patches;
- the launch command and fully resolved configuration;
- model, reward, dataset, and tokenizer revisions or content hashes;
- GPU model/count, node/interconnect description, driver, CUDA, PyTorch, generation-engine, and training-backend versions;
- raw stdout/stderr, per-rollout structured timing, memory and utilization traces, checkpoint metadata, and evaluation outputs;
- seed, warm-up exclusion rule, failure/retry accounting, and statistical aggregation script.

For training runs, serialized `Sample` traces must redact private prompt or media payloads while retaining lineage IDs, tensor shapes and dtypes, behavior-policy versions, reward components, and replay-segment index metadata. For system baselines, an alignment manifest must explain every nonidentical setting and whether it changes effective work.

D Correctness status and remaining checks

Executed locally. The preliminary RQ3 harness performs ten fail-fast checks on the audited source: lineage-valid cross-stage construction; exact split/concat; tree-complete select and slice; mean reward propagation plus group-normalized advantages; rejection of bad lineage, modality keys, and primitive batch sizes; packed variable-length text select/concat; and lazy tensor-reference selection, materialization, identity, and bounds behavior. The structured outcomes and data-sensitive SHA-256 round-trip hash are stored with the run artifacts.

Required before complete evidence admission. The following items specify the system-level verification gates required for full evidence admission, followed by the current status established by the `2026-10-08-e0-e1-e2-e4` bundle and accompanying execution records. While the bundle archives the exported E0 parity gauge measurements (`e0_parity.json`), the E0 harness code and raw smoke-run logs reside outside this repository and were not re-executed locally.

1. **Rollout–replay parity:** before the first update, compare engine-emitted and train-side replay log probabilities or transition statistics against prespecified numerical tolerances. *Status:* measured as diagnostic magnitudes without an enforced acceptance threshold. On E0 smoke runs, the pre-update maximum absolute log-probability difference is at most 1.0×10^{-5} across the diffusion paths, and the mean absolute difference is 0.0152 on the autoregressive path (Table 18), recorded under a non-blocking threshold of 10.0. Because the diffusion paths average each transition's log-density over all latent elements, element-averaged differences at this scale can mask localized conditioning mismatches (Section 7.1). At the full E1 geometry (32 GPUs, 8,192 maximum new tokens), four pre-launch probes recorded 37 parity checks (4 pre-update and 33 post-publication), with per-rollout mean differences of 0.0046–0.0177 and maximum single-token differences of 0.32–3.49. Those probes configured a nominal threshold of 0.05 on the single-token maximum with enforcement disabled, which every check exceeded; the operator sign-off inspected the per-rollout mean instead, for which no threshold had been prespecified. Consequently, these measurements record empirical discrepancy magnitudes rather than passes against a formal gate.
2. **Publication parity:** repeat the rollout–replay parity check after weight publication for every LoRA or full-weight synchronization handler used in an experiment. *Status:* measured post-publication for five of the six synchronization configurations in E0 smoke runs—reaching at most 1.1×10^{-5} on the diffusion paths (Table 18)—and after every weight publication across the six formal E1 runs and the seed 11 rerun (5,318 checks in total; Table 19). Across the E1 runs, the per-rollout mean absolute difference has a run-level median of 0.0032–0.0079 and a maximum of 0.044, whereas the single-token maximum reaches 30.0. The formal E1 runs logged this gauge without a configured tolerance; relative to the nominal 0.05 probe reference, every per-rollout mean falls below 0.05 while every single-token maximum exceeds it. No formal acceptance threshold has been established for either statistic, and no experiment in this report uses the sixth configuration (`CheckpointWeightSync`).
3. **Batch-version integrity:** verify that an optimizer batch never mixes behavior versions when the configured algorithm assumes a single behavior policy; although the asynchronous, agentic, and native-vLLM weight-transfer paths all increment v_b upon weight publication, mixed-version batches can arise at runtime only on the asynchronous path, where multiple behavior versions may be in flight simultaneously. *Status:* enforced in code by the asynchronous batch assembler at the commits inspected in E0; no dedicated runtime trace is archived in the bundle, and no asynchronous training results are reported from this bundle.
4. **Boundary integrity:** ensure that checkpointing and evaluation begin only after the asynchronous rollout manager drains in-flight trajectories, and that job resumption restores the optimizer version and republishes weights before generating new rollouts. *Status:* enforced in code at the commits inspected in E0; no dedicated verification trace is archived.
5. **Work accounting:** compare generated tokens or pixels, denoising and replay steps, scored samples, and committed optimizer updates across systems. *Status:* not shown. No cross-system count comparison is archived with these runs.

6. **Structural preconditions:** validate unique root IDs, uniform layer depth, parent-major contiguity, shared-field equality, and behavior-version provenance before merging batches or propagating credit, and verify equivalence against an ID-joined flat-row reference. *Status:* both grouping scopes match an independent ID-joined reference on synthetic fixtures (E4a) and on recorded rollouts from the composed training runs (E4 v2, Section 8.3). Static and test inspection in the E4 audit notes that group-advantage computation explicitly rejects non-contiguous (interleaved) group rows, whereas leaf-to-root reward propagation assumes parent-major order without raising an error if rows are interleaved.

Table 18: E0 rollout–replay parity on smoke runs: absolute difference between rollout-engine and train-side replay log probabilities before the first optimizer update and after one weight publication. Each cell represents a single smoke check. `CheckpointWeightSync` is unsupported by the `vllm_omni` receiver used in that row, so only its pre-update check is recorded; the in-process `train_engine` rollout mode shares model weights directly and therefore performs no weight publication. No formal acceptance tolerance has been set.

		Before first update		After publication	
Handler	Recipe and topology	max	mean	max	mean
Diffusion smoke runs (two samples per prompt)					
`CheckpointWeightSync`	`sd3_vllmomni_full_tensor` with the handler overridden; colocated, 8 GPUs	7.2×10^{-6}	4.3×10^{-6}	–	–
`IPCWeightSync`	`sd3_vllmomni_full_ipc`; colocated, 8 GPUs	7.8×10^{-6}	3.8×10^{-6}	1.0×10^{-5}	4.4×10^{-6}
`LocalLoraWeightSync`	`sd3_vllmomni`; colocated, 8 GPUs	1.0×10^{-5}	3.0×10^{-6}	1.1×10^{-5}	4.2×10^{-6}
`NCCLWeightSync`	`sd3_vllmomni_full_nccl_separate`; separate, 4 + 4 GPUs	5.4×10^{-6}	3.3×10^{-6}	8.7×10^{-6}	3.9×10^{-6}
`RemoteLoraWeightSync`	`sd3_vllmomni_lora_separate`; separate, 4 + 4 GPUs	5.5×10^{-6}	2.3×10^{-6}	4.5×10^{-6}	2.6×10^{-6}
`TensorWeightSync`	`sd3_vllmomni_full_tensor`; colocated, 8 GPUs	6.6×10^{-6}	4.9×10^{-6}	8.5×10^{-6}	4.3×10^{-6}
Train-side engine	`sd3_trainside`; colocated	7.3×10^{-6}	1.7×10^{-6}	–	–
Autoregressive smoke runs (8 GPUs, 256 new tokens)					
`TensorWeightSync`	`qwen3_grpo_4b_base_dapo_sglang`; colocated with SGLang, 8 GPUs	0.473	0.0152	0.499	0.0188

Table 19: Rollout–replay parity after every weight publication in the six formal E1 runs (800 rollouts each) and the seed 11 rerun (518 rollouts), all with `TensorWeightSync` to SGLang on 32 GPUs. For each rollout the gauge records the mean and the maximum absolute log-probability difference over tokens; the table gives the median, 95th percentile, and largest value of each over a run's rollouts. No acceptance tolerance has been set.

	Mean over tokens			Maximum over tokens		
Run	median	95th pct.	largest	median	95th pct.	largest
GRPO, seed 11	0.0032	0.0070	0.0193	1.40	4.04	18.89
GRPO, seed 11, rerun	0.0042	0.0076	0.0169	1.38	3.23	30.04
GRPO, seed 22	0.0036	0.0077	0.0187	1.35	3.51	22.55
GRPO, seed 33	0.0037	0.0081	0.0175	1.42	3.21	24.36
DRPO, seed 11	0.0078	0.0115	0.0442	1.07	2.68	18.33
DRPO, seed 22	0.0067	0.0107	0.0392	1.11	3.06	9.67
DRPO, seed 33	0.0079	0.0104	0.0385	0.93	2.17	6.20

E Additional E1 evidence

Internal validation and validation-selected checkpoints. During training, the E1 protocol evaluates all 60 AIME 2024/2025 problems every ten rollouts (16 samples per problem under the training decoding parameters) and treats any checkpoint chosen by this curve strictly as validation-selected (Table 20 and Figure 11a). In every formal run, validation accuracy peaks at or before rollout 400 and declines by rollout 800. The three formal GRPO seeds finish between 0.001 and 0.022 (compared with 0.015–0.031 before training), whereas the three DRPO seeds finish between 0.144 and 0.159—below their respective peaks of 0.222–0.248, but still above the highest validation accuracy reached by any formal GRPO seed. Restricted

to the four saved checkpoints (rollouts 200, 400, 600, and 800) of each formal run, validation accuracy selects rollout 200 in five runs and rollout 400 in DRPO seed 11. For GRPO seeds 22 and 33, these validation-selected checkpoints achieve 0.8145 and 0.7915 on MATH-500—exceeding the frozen base (0.6695) even though their final rollout-800 checkpoints fall below it—while for the three DRPO seeds they achieve 0.8675, 0.8710, and 0.8645, each surpassing the corresponding rollout-800 checkpoint. Because internal validation evaluates the AIME 2024/2025 benchmark problems directly, we do not report external AIME test scores for validation-selected checkpoints, and the rollout-800 checkpoint remains the prespecified primary endpoint. In GRPO seed 11, the two lost checkpoints (rollouts 200 and 400) had higher validation accuracy than rollouts 600 and 800, so Table 10 captures only the degraded second half of that run and its rollout-200 validation-selected MATH-500 score is unavailable. Because generation sampling was unseeded, the GRPO seed 11 rerun follows a distinct trajectory despite consuming the same prompts in the same order: its validation accuracy exceeds the original run's at every evaluation from rollout 70 to 420 (reaching 0.131 versus 0.082 at rollout 400) before falling below the original run from rollout 430 until termination (0.004 versus 0.049 at rollout 510). Finally, in four of the six formal runs the overall validation peak occurs between multiples of 200 rollouts, where the 200-rollout checkpoint cadence did not save model weights.

Table 20: E1 internal validation and validation-selected checkpoints. Validation accuracy is the share of 960 sampled answers graded correct on the 60 AIME 2024/2025 problems; per-problem results are not exported, so no sampling error is given. The selected checkpoint is the saved checkpoint with the highest validation accuracy, and its MATH-500 score is the avg@4 of Table 10. The seed 11 rerun has two saved checkpoints and no rollout 800. †Mean of two independent evaluations.

	Validation accuracy			Validation-selected checkpoint			Final
Run	before	highest (rollout)	at 800	rollout	validation	MATH-500	MATH-500
GRPO, seed 11	0.0208	0.1260 (280)	0.0177	200	0.0979	lost	0.5920†
GRPO, seed 11, rerun	0.0188	0.1823 (330)	–	400	0.1312	0.7875	–
GRPO, seed 22	0.0312	0.1333 (160)	0.0010	200	0.1115	0.8145	0.3030
GRPO, seed 33	0.0146	0.1177 (200)	0.0219	200	0.1177	0.7915	0.6315
DRPO, seed 11	0.0250	0.2271 (400)	0.1594	400	0.2271	0.8675	0.8488†
DRPO, seed 22	0.0208	0.2219 (260)	0.1438	200	0.2135	0.8710	0.8120
DRPO, seed 33	0.0344	0.2479 (160)	0.1510	200	0.2156	0.8645	0.8240

Answers at the token limit. Table 21 reports the fraction of MATH-500 completions that reach the 16,384-token generation limit at each evaluated checkpoint, derived from an offline regrade of the stored completions that exactly reproduces every avg@4 score in Table 10. In the three formal GRPO runs, completions that reach the token limit are rarely graded correct (at most 6.4%), so penalizing non-terminating completions as incorrect alters their MATH-500 scores only slightly. By contrast, at DRPO seed 22's final checkpoint, 74.9% of all MATH-500 completions (1,498 of 2,000) reach the token limit and 75.3% of those (1,128 of 1,498) are still graded correct because `math_verify` extracts boxed answers prior to the cutoff; scoring limit-hitting completions as incorrect lowers that checkpoint's MATH-500 avg@4 from 0.8120 to 0.2480. The same checkpoint reaches the generation limit on 479 of 480 completions on AIME 2024 and 476 of 480 on AIME 2025, so its AIME entries in Table 10 depend on the same lenient grading behavior.

Table 21: E1 answers at the token limit on MATH-500. An answer is at the limit when its stored text, tokenized again, has at least 97% of the 16,384-token generation limit. Columns 200 to 800 give the share of a checkpoint's answers at the limit. For the final checkpoint the table gives how many answers at the limit are graded correct, the share of the other answers graded correct, and avg@4 with every answer at the limit counted wrong. Seed 11's final checkpoints pool their two evaluations; the seed 11 rerun has no final checkpoint.

	Answers at the limit, by rollout				Final checkpoint		
Run	200	400	600	800	correct at the limit	correct elsewhere	limit counted wrong
Frozen base			3.0%		6 of 59	0.687	0.6665
GRPO, seed 11	–	–	19.8%	10.6%	17 of 425	0.658	0.5878
GRPO, seed 11, rerun	6.2%	9.9%	–	–	–	–	–
GRPO, seed 22	9.4%	13.8%	20.0%	30.3%	39 of 606	0.407	0.2835
GRPO, seed 33	4.6%	9.4%	10.1%	20.0%	25 of 399	0.773	0.6190
DRPO, seed 11	3.8%	7.8%	0.6%	0.1%	1 of 5	0.850	0.8485
DRPO, seed 22	1.2%	9.7%	1.6%	74.9%	1,128 of 1,498	0.988	0.2480
DRPO, seed 33	0.4%	0.1%	0.6%	0.7%	1 of 14	0.829	0.8235

Training diagnostics. Figure 11 plots the exported per-rollout E1 training diagnostics (the policy-loss and wall-clock series specified in the protocol were not exported). Across all three formal GRPO seeds between the first 100 rollouts and the last 100 rollouts, mean response length increases from 1,614–2,276 to 4,074–4,483 tokens, the fraction of completions reaching the 8,192-token training cap rises from 3.6%–9.1% to 15.0%–30.5%, and the fraction of zero-variance prompt groups (where all eight samples receive identical reward and contribute zero gradient) rises from 46%–48% to 62%–80%. Meanwhile, step-size diagnostics remain small: across 25-rollout blocks in all six formal runs, the median approximate KL divergence is at most 0.0014, the median maximum importance ratio is at most 2.9, and no formal GRPO rollout clips more than 2.4% of its tokens, although isolated rollouts spike up to an approximate KL of 0.032 and a maximum importance ratio of 62.9. The GRPO degradation thus coincides with response lengthening, increased truncation, and shrinking within-group reward variance rather than importance-ratio instability or excessive clipping; however, the DRPO runs reach even longer mean response lengths (Figure 11b), showing that response growth alone does not explain why GRPO degrades while DRPO improves.

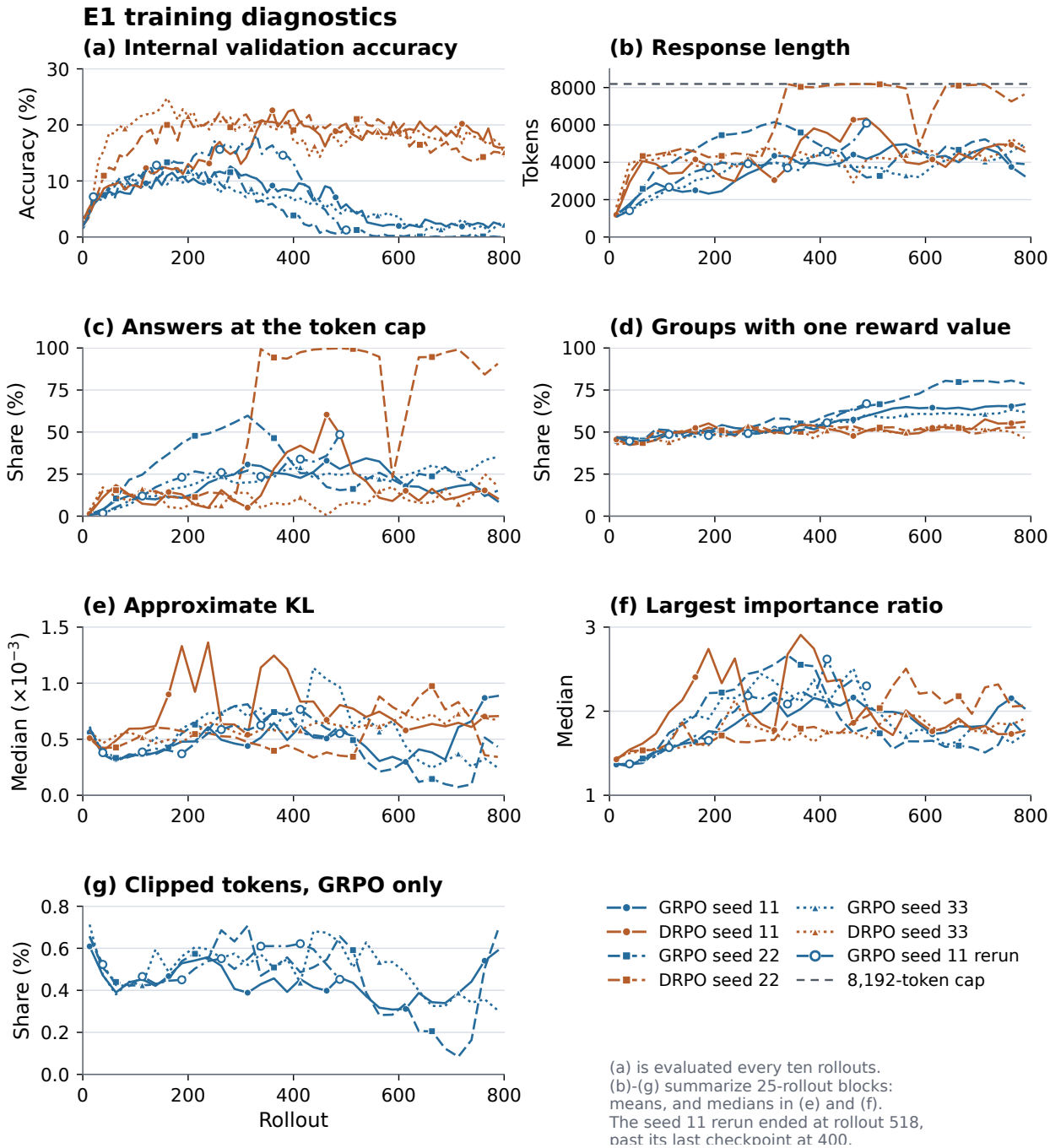


Figure 11: E1 training diagnostics for the six formal runs and the GRPO seed 11 rerun (which terminated at rollout 518 and is plotted with hollow markers). (a) Internal validation accuracy on AIME 2024/2025, evaluated every ten rollouts. (b)–(g) Per-rollout training metrics aggregated over consecutive 25-rollout blocks, reporting block means in (b)–(d) and (g) and block medians in (e) and (f), where individual rollouts exhibit spikes; the rerun's block curves end at its last complete 25-rollout block (rollout 500). Because the DRPO objective has no clipping term, (g) plots GRPO runs only. Color encodes the recipe arm, while marker shape and line style encode the seed.

Discarded launches and checkpoint provenance. The following aborted or superseded segments are excluded from Table 10: three initial launches of GRPO seed 11 that failed prior to rollout 1; an initial launch of GRPO seed 22 stopped at rollout 19 when its compute allocation was reassigned to the DRPO arm; an initial launch of DRPO seed 11 that ran for 39 rollouts with identically zero gradients before overriding `rl_on_policy_target=null` (an override retained by all three reported DRPO runs); and rollouts 201–369 from DRPO seed 22's first allocation prior to its resumption from checkpoint 200. The GRPO seed 11 rerun executed the formal 800-rollout launch command, was stopped after saving and exporting checkpoint 400, and was then resumed from checkpoint 400 under the same command until its allocation expired at rollout 518 (prior to checkpoint 600); in the exported training series, the single post-400 rollout logged before the stop is replaced by the resumed segment's rollout, and the duplicate initial validation evaluation logged upon checkpoint restoration is omitted. Finally, each external benchmark evaluation ran against a bf16 export of the saved checkpoint without recording a cryptographic hash of the exported weights, so the bundle identifies evaluated checkpoints by run ID and rollout step alone.